

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2023.0322000

E-XAI: Evaluating Black-Box Explainable AI Frameworks for Network Intrusion Detection

OSVALDO ARRECHE¹, TANISH R. GUNTUR², JACK W. ROBERTS³ and MUSTAFA ABDALLAH³
(Member, IEEE)

¹Electrical and Computer Engineering Department, Purdue School of Engineering and Technology, Indiana University-Purdue University Indianapolis (IUPUI), Indianapolis, IN, USA (e-mail: oarreche@iu.edu)

²Computer and Information Science Department, Indiana University-Purdue University Indianapolis (IUPUI), Indianapolis, IN, USA (e-mail: tguntur@iu.edu)

³Computer and Information Technology Department, Purdue School of Engineering and Technology, Indiana University-Purdue University Indianapolis (IUPUI), Indianapolis, IN, USA (e-mails: roberjaw@iu.edu, mabdall@iu.edu)

Corresponding author: Mustafa Abdallah (e-mail: mabdall@iu.edu).

This work is supported in part by AnalytiXIN, Enhanced Mentoring Program with Opportunities for Ways to Excel in Research (EMPOWER), and 1st Year Research Immersion Program (1RIP) grants from the office of the Vice Chancellor for Research at Indiana University-Purdue University Indianapolis.

ABSTRACT The exponential growth of intrusions on networked systems inspires new research directions on developing artificial intelligence (AI) techniques for intrusion detection systems (IDS). In particular, the need to understand and explain these AI models to security analysts (managing these IDS to safeguard their networks) motivates the usage of explainable AI (XAI) methods in real-world IDS. In this work, we propose an end-to-end framework to evaluate black-box XAI methods for network IDS. We evaluate both global and local scopes for these black-box XAI methods for network intrusion detection. We analyze six different evaluation metrics for two popular black-box XAI techniques, namely SHAP and LIME. These metrics are descriptive accuracy, sparsity, stability, efficiency, robustness, and completeness. They cover main metrics from network security and AI domains. We evaluate our XAI evaluation framework using three popular network intrusion datasets and seven AI methods with different characteristics. We release our codes for the network security community to access it as a baseline XAI framework for network IDS. Our framework shows the limitations and strengths of current black-box XAI methods when applied to network IDS.

INDEX TERMS XAI Evaluation, Intrusion Detection Systems, SHAP, Explainable AI, Network Security, LIME, Black-box AI, NSL-KDD, CICIDS-2017, and RoEduNet-SIMARGL2021.

I. INTRODUCTION

The power of artificial intelligence (AI) has inspired the development of fully automated intrusion detection systems (IDS) [1], [2]. There are several AI techniques that have been leveraged for such automation of intrusion detection, including neural networks [3], [4], support vector machine [5], [6], decision trees [7], [8], naive bayes [9], [10], and random forest [11], [12]. However, most of these AI methods (except decision trees for simple IDS) are black-box models where no explanations about the AI's decisions are provided to the security analyst managing these IDS [13], [14]. In particular, these previous AI-based studies focused more on the classification accuracy of various AI algorithms without providing insights about their behavior and reasoning. Such a limitation motivates the pressing need to leverage the relatively recent explainable AI (XAI) field for enhancing the understandability of AI decisions in IDS [15].

There are several recent works that started exploring

leveraging XAI for IDS (e.g., [16]–[20]). The work [19] proposed an abstract design for a human-in-the-loop approach for intelligent IDS. Moreover, the work [16] used only a simple decision tree algorithm for extracting decision rules using exclusively the NSL-KDD benchmark dataset [21]. The work [17] incorporated the local interpretable model-agnostic explanations (LIME) [22] to only provide local explanations for a single AI model (SVM) for the used IDS. The work [18] infused the CIA principle (“C” stands for confidentiality of information, “I” stands for integrity of data, and “A” stands for availability) for enhancing the explainability of the AI model on the CICIDS-2017 benchmark intrusion dataset. However, all of these works did not provide a thorough evaluation of XAI methods under different intrusion datasets and diverse sets of AI models which affects the generality of such works. Although XAI-based tools are going to be ubiquitous in network security to aid security analysts to be more efficient

and accurate in their network defense tasks, one main challenge of deploying XAI tools into network intrusion detection is validating such tools, testing their quality, and evaluating the relevant security metrics. Such challenges decrease the trust in using XAI methods for real-world deployment in network IDS.

This paper aims to fill such a gap in evaluating XAI methods for network intrusion detection systems. We build a framework for evaluating XAI methods when applied to network intrusion detection. We build on the prior work [23] that delineates six metrics to evaluate XAI methods while adapting these metrics for network IDS. These metrics are descriptive accuracy, sparsity, stability, efficiency, robustness, and completeness. These metrics are defined as follows.

- **Descriptive Accuracy:** The descriptive accuracy can be interpreted as the importance of top intrusion features when removed from the AI model. For instance, if an intrusion feature is important, it is expected that the intrusion prediction (by the AI model) will be less accurate if such a feature is taken out. If the accuracy drop after the feature is taken out is high, then the XAI explainability power of such a feature is high.
- **Sparsity:** The sparsity measures the correlation between the importance scores, given by the XAI method, of different features. This is measured by comparing how many features are below a certain small threshold. For example, if 18 out of 20 intrusion features are below a small threshold (close to zero), this means that there are 2 features with extremely high influence on the model's decision, and therefore the sparsity is high. Having an XAI method with high sparsity can help security analysts monitor their network traffic by monitoring the lower number of traffic metrics.
- **Efficiency:** The efficiency of an XAI method can be defined as how much time the XAI method takes to generate an explanation. This metric is important because it measures the applicability of the XAI method in real-world systems since it is preferable that the explanations are generated quickly rather than slowly for practical purposes. As the ultimate goal is to aid security analysts, the expectation is to be able to generate accurate XAI explanations in real-time to detect intrusions in a timely manner.
- **Stability:** The stability metric measures how consistent the XAI method is in generating its explanations. This is measured by checking how many features are common among different running trials with the same conditions. Based on such stability test, an XAI method with higher stability can be trusted more by the security analyst in the network intrusion detection process.
- **Robustness:** The robustness of an XAI method can be defined as the ability of that XAI method to generate the same explanations under small perturbations in the intrusion features. Such perturbation can be due to computational errors or an adversarial attack. For this work, we adapted the adversarial model of the work [24].

This adversarial model consists of training an extremely biased model that will rely only on one feature to base its prediction on and training another model with all the features plus a new feature that is engineered to fool the XAI explanation method. Based on such two trained models, XAI method under test will generate an explanation of a normal sample that is convincing to a security analyst while in reality the real sample was hidden from the XAI method. This can compromise the integrity of the framework via disconnecting the explanation from the underlying behavior, and thus disguise an attack as normal network traffic.

- **Completeness:** The completeness characteristic of an XAI method refers to the ability of the XAI method to provide a correct explanation for every possible network traffic sample, including corner cases. If the XAI method is not complete, this opens the door for the intruders to exploit and trick the XAI method to create degenerated results. We emphasize that if an XAI is complete, it automatically becomes more robust (i.e., its reliability for the end user is increased because it can detect whether the explanation is valid). Nonetheless, while the measure for completeness is by checking that every sample has a valid explanation, the robustness metric in this work refers to how resistant an XAI framework is when facing an adversarial attack.

We explore the aforementioned six evaluation metrics on two popular black-box XAI methods. These two XAI methods are LIME [25] and SHAP [26]. LIME and SHAP are well-known and widely-used XAI techniques by the network intrusion research community [23], [24], [27], [28]. LIME is used within the scope of local explanations, and is based on linear approximation generated by creating perturbations around the sample to understand the behavior of the inputs for a given decision generating rules on how that model behaves on that sample. On the other hand, SHAP has both local and global scopes. This method is based on the Shapley value concept from game theory where it measures the importance of each feature by calculating its corresponding Shapley value. Then, if the AI model performs the same without such a feature, it will have a low importance.

We explain the step-by-step process for each one of our six evaluation metrics (descriptive accuracy, sparsity, efficiency, stability, completeness, and robustness) to generate our XAI evaluation metrics and their results.

We evaluate our framework on three network intrusion datasets with different characteristics. The first dataset is the recent RoEduNet-SIMARGL2021 dataset [29] collected from the SIMARGL project (supported by the European Union). It contains realistic network traffic along with features that can be calculated from live traffic, which makes the dataset highly usable for network intrusion detection systems. The second dataset is CICIDS-2017 [30] which is a benchmark intrusion detection dataset created by the Canadian Institute for Cybersecurity at the University of Brunswick in 2017 with different attack profiles. The final

dataset is the NSL-KDD dataset [21], which is widely recognized for network intrusion detection systems. For each dataset, we evaluate the two XAI approaches (SHAP and LIME) for different black-box AI models where we present and discuss the results of the six evaluation metrics generated for each XAI method. We also categorize AI models according to their performances on XAI evaluation metrics with the two XAI methods (SHAP and LIME) considered in this work. In this context, we provide discussions about possible usage of different AI models given the XAI method and the related XAI evaluation metric. We furthermore perform a statistical analysis to validate our findings. We apply the non-parametric Wilcoxon signed-rank statistical test to the three datasets used in this work.

This work represents a step towards closing the gap of applying black-box XAI methods for network IDS via applying evaluation and comparison metrics for testing these XAI methods. These metrics include network security requirements (e.g., stability, robustness, and efficiency) and general properties of AI models (e.g., descriptive accuracy, sparsity, and explainability). Thus, our framework would expand the application of XAI into network intrusion detection systems, paving the way for more visibility and enhancement for this research area.

Summary of Contributions: We summarize below our main contributions.

- We propose an end-to-end framework for evaluating XAI techniques for network intrusion detection tasks. Our framework evaluates both global and local explanations for XAI methods for network intrusion detection.
- We analyze six different evaluation metrics for two popular black-box XAI techniques, namely SHAP and LIME.
- We evaluate our XAI evaluation framework using three network intrusion datasets and under seven different AI models.
- We release our source codes for the community to access it as a baseline XAI evaluation framework for network intrusion detection and to build on it with new datasets and models.¹

II. RELATED WORK

Existing Efforts in Leveraging XAI for IDS: The survey [33] offers an overview of ongoing efforts in introducing a framework for the development of Explainable AI-based Intrusion Detection Systems (XAI-IDS), accompanied by a taxonomy for standardizing concepts. This survey builds upon the distinctions between black-box models (opaque models) and white-box models (transparent models), examining their trade-offs in terms of explainability and performance.

In this context, several recent works have begun exploring the utilization of Explainable Artificial Intelligence (XAI)

for intrusion detection systems (IDS) [16]–[19]. For instance, [16] employed decision tree algorithms to extract rules in IDS using the NSL-KDD dataset. The work [17] incorporated LIME to provide local explanations for an SVM model in the IDS. The authors in [18] integrated the CIA principle to enhance the explainability of AI models using the CICIDS-2017 dataset. In summary, most of these efforts can be found in [33], which maps the current state-of-the-art works in the field. The paper [33] also proposes an explainable framework for IDS. This framework consists of a ‘pre-processing phase’ which involves taking a raw dataset as input and outputting a high-quality dataset. The second step in the framework is the ‘modeling phase’ where the high-quality dataset is used to generate explanations for the results. The last step is the ‘post-modeling explainability Phase’ which is characterized by the user interface that can be customized for its audience.

There are also other notable related works, including the work [20] that introduces a new framework named OWAD (Open World Anomaly Detection), which pioneers in addressing normal concept drift in deep learning-based models. Similarly, the work [31] contributes a new framework called DeepAID, aiming to address the limitations of unsupervised DL-based anomaly detection systems. Shifting the focus to attacks in camera network systems, the work [32] showcases an online (i.e., real-time) anomaly detection system for network intrusion detection. Table 1 shows the differences between our work and these prior works.

XAI Evaluation: There are few works that considered evaluating XAI methods for network intrusion detection [23], [24]. The work [23] has established six metrics to evaluate XAI methods for applying deep learning in security. They used these metrics to evaluate black-box and white-box XAI methods for two datasets Malware Genome Project and CWE-119. This paper compared the performances of white-box explanation methods (IG, GRADIENTS, and LRP) [23] and black-box XAI methods (SHAP, LIME, and LEMNA). Although the realm of [23] lies in the security domain, their focus was on different subsets of the security domain. In particular, the CWE-119 dataset consists of malicious code samples that take advantage of improper use of memory buffer which can help programmers in finding vulnerabilities in code implementation. On the other hand, the Malware Genome Project contains and systematizes several Android malware samples. Therefore, these applications differ from our work’s focus on network intrusion detection. The paper [24] has developed an adversarial attack on LIME and SHAP to test their robustness. The main idea is to train two different models, one that is extremely biased and its decision is based on one feature, and another one that is innocuous with an extra ‘unrelated’ feature that is inserted to have a direct correlation with the prediction results. Then the innocuous prediction is passed into LIME and SHAP to deceive them to create seemingly correct explanations although the associated sample is an attack. In this work, the authors used three different datasets which

¹The URL for our source codes is: https://github.com/ogarreche/XAI_metrics

TABLE 1: A comparison between different aspects of our own work and prior relevant works on XAI for network intrusion detection (including methods for explanations, datasets, and AI models).

Paper	Dataset	Model	Explainer	Evaluating XAI
Our Work	CICIDS-2017, NSL-KDD, RoEduNet-SIMARGL2021	LGBM, DNN, MLP, CNN, RF, SVM, ADA, KNN	SHAP, LIME	Yes
DeepAID [31]	Kitsune-Mirai, HDFS, CICIDS-2017, LANL-CMSCSE	DNN, LSTM, GNN Autoencoders, Kitsune	GLGV, DeepLog, DeepAID	No
Kitsune [32]	Kitsune (OS Scan, Fuzzing, and Mirai)	Kitnet, GMM, SVM, DNN, Autoencoders	Kitsune	No
OWAD [20]	Kyoto 2006+, BGL, LANL-CMSCSE	DNN, Autoencoder, DeepLog, APT (GLGV), GAN, LSTM	OWAD, CADE, TRANS	No
Feature-oriented Design [28]	NSL-KDD	CNN, DNN	LIME, and Saliency View	No
Explainable ML Framework [27]	NSL-KDD	DNN, RF, SVM, KNN, ResNet50	SHAP	No
fooling LIME and SHAP [24]	COMPAS, German Credit, Communities and Crime	RF	SHAP, LIME	No
Evaluating XAI in Security [23]	Malware Genome Project, CWE-119	CNN, MLP, RNN	SHAP, LIME, LEMNA, IG, LRP, Gradients	Yes

are COMPAS, Communities and Crime, and German Credit. However, these datasets are outside the security domain. In particular, COMPAS contains information about criminal history, prison time, and demographic attributes. Similarly, the Communities and Crime dataset contains socioeconomic and crime information for different communities. The last dataset (German Credit) contains information on financial and demographic loan applicants. The authors shared their code for the public and we adapted it to our network intrusion detection context for testing the robustness metric of the XAI methods considered in this work.

Contribution of This Work: In contrast to the works [23] and [24], we propose an end-to-end XAI evaluation framework that is tailored to network intrusion detection. We evaluate our framework on new datasets that are not considered in these prior works. We also consider a diverse set of AI models while considering the six XAI evaluation metrics. We present the analysis for descriptive accuracy and sparsity using both black-box XAI methods across all models and datasets to check the importance of the top features chosen by each XAI method. Then, through repeated trials, we generate a stability analysis for the two XAI methods. We then analyze the computational efficiency of each XAI method, checking its viability for real-world deployment. We emphasize that we consider such metrics for both global and local scopes. Furthermore, we generate a few attacks to test the robustness and completeness of the two XAI techniques. Our end-to-end XAI evaluation framework presents an essential first step towards measuring main metrics in deploying XAI for network IDS.

III. THE PROBLEM STATEMENT

We now provide the main preliminaries for network intrusion detection, the challenges of black-box AI, the need for XAI, and the challenges in evaluating XAI methods when applied to network intrusion detection.

A. NETWORK INTRUSION TYPES

There are several common network intrusion types. In our work, we consider the main network attacks in the common MITRE ATT&CK framework [34]. Thus, the network traffic

can be divided into the following categories:

Normal traffic: is the regular traffic collected from the network.

PortScan (PS) / Network Service Discovery [MITRE ATT&CK ID: T1046]: It is an intrusion in which the attacker aims to make recognizance of the victim's computer. It is often used as the first step of an attack to search for vulnerable points and possible entrance ports. The functionality is to send a connection solicitation to the victim without ever finalizing the connection. However, for this attack, the solicitations are sent to various ports and the ones that send a message back are mapped as possible entrance points [35].

Denial of Service (DoS) / Network Denial of Service [MITRE ATT&CK ID: T1498]: It is a type of attack where the objective is to render the target unavailable for the network. One popular example of such an attack is where the attacker keeps sending solicitations to connect with a server. However, when the server accepts the solicitation and sends an acknowledgement to the origin expecting its response it never receives a response. As a result, the memory for these open communications is left open and fully consumed until the server becomes unavailable. We refer to [36], [37] for detailed classes of DoS attacks.

Brute Force [MITRE ATT&CK ID: T1110]: It is an attack in which all password possibilities are attempted by the intruder in order to break into the victim's network. This attack is often paired with the knowledge of the most common used passwords. It can become effective in cases in which a user has a weak or a common easy-guessing password [36].

Web Attack / Initial Access [MITRE ATT&CK ID: TA0001, T1659, T1189]: It is a class of attacks performed via the web in which the attacker exploits web vulnerabilities. For example, an attacker may gain access to the application's underlying instance or container by exploiting a public-facing application, using a software bug, misconfiguration, or glitch. The web attack can also include attacks such as the Drive-by Compromise [38], however web attacks (e.g., SQLi, XSS) typically do not provide initial access to a remote server [34].

Infiltration / Initial Access [MITRE ATT&CK ID: TA0001]: This attack occurs when someone tries to get initial access to a system or application. Such class of attacks consist of a myriad of techniques as targeted spearphishing and exploiting weaknesses on public-facing web servers. The foothold gained by this attack can range from a simple change of password to continue access through valid accounts and external remote services.

Botnet / Compromise Infrastructure [MITRE ATT&CK ID: T1584.005, T1059, T1036, T1070]: It is an automated class of attacks which is performed by hijacked devices executed remotely by the attacker in which scripts (bots) mimic human behaviour and duplicate it [39]. This scripted technique allows scalability and easy deployment making it an ideal tool to touch multiple attack points simultaneously. Thus, Botnet is a very common network attack type.

Probe Attack/ Network Scanning or Surveillance [MITRE ATT&CK ID: T1595]: Probe attacks are often the preliminary stage of a more comprehensive attack. These attacks involve scanning a network to gather information or find known vulnerabilities [40]. An attacker with a map of machines and services that are available on a network can use this information to look for exploits. We emphasize that while port scanning is a form of probe attack, not all probe attacks are port scans. Some might target specific vulnerabilities or use different methods. Examples include ping sweeps [41], and DNS zone transfers [42].

Remote to Local Attack (R2L) [MITRE ATT&CK ID: TA0001, T1110, T1078]: R2L is class of attacks in which an attacker (who can send packets to a machine over a network but does not have an account on that machine) gains unprivileged access as a user of that machine (i.e., initial access). Once the attacker has user-level access, they may attempt to escalate privileges to gain control over the entire system (i.e., U2R attack).

User to Root Attack (U2R) [MITRE ATT&CK ID: TA0004, T1078]: User to Root attack is a type of exploit in which the attacker starts out by accessing an unprivileged user account on the system (via sniffing passwords, a dictionary attack, or social engineering) to exploit some vulnerability to gain root access to the system. This can be extremely detrimental as it can allow the attacker to manipulate the system as a root user or an administrator. Note that R2L is a necessary preliminary for Privilege Escalation (i.e., U2R).

B. INTRUSION DETECTION SYSTEMS

The increasing sophistication of network attacks poses a significant threat to critical infrastructure across various sectors [43], [44]. Consequently, IDS plays a crucial role in safeguarding computer network systems against malicious activities, whether initiated by internal users or external infiltrators [45]. Traditional IDS designs typically rely on the assumption that an intruder's behavior will deviate noticeably from that of a legitimate user, making many unauthorized actions detectable [46]. With recent advancements in AI over

the past decade, this design paradigm has paved the way for the development of AI models capable of automatically detecting network intrusions [47].

C. SHORTCOMINGS OF BLACK-BOX AI MODELS

While AI models have greatly automated intrusion detection, they possess a black-box nature due to the intricate relationships between features that underlie their decision-making processes. This inherent complexity makes it challenging to comprehend how and why these models arrive at particular results. The black-box issue is pervasive across many AI models, including random forests and deep neural networks. Despite their high predictive accuracy in IDS, explaining the rationale behind their behavior, especially when errors occur, can be elusive. This becomes especially critical in safety-sensitive applications such as network security through IDS. Consequently, there is a growing motivation to provide explanations for the decisions made by AI models in the context of IDS [48].

D. MAIN CATEGORIES OF EXPLAINABLE AI (XAI)

Explainable AI is a relatively recent research field that has been developed to address the aforementioned black-box nature of most AI models. The explainability in AI can be considered as the collection of features of the interpretable domain that have contributed to a given example to produce a decision by the AI model in a task (e.g., classification or regression). Without loss of generality, XAI can be divided into the following subareas: (1) XAI techniques that explain the behavior of an AI model globally (on many data instances) or locally (on specific instances), (2) XAI techniques that are model-agnostic (can explain any AI model generally) or model-specific (tailored for explaining specific AI models), and (3) XAI explanation methods which can be either the creation of a surrogate model (i.e., the creation of a simple model to explain a complex one) or the creation of a feature importance list for the global AI model (i.e., this is based on Shapley values, or other available methods).

E. BENEFITS OF XAI FOR NETWORK IDS

The output generated by the current AI-based IDS often poses challenges for human analysts, as it is not readily interpretable. Consequently, analysts are required to sift through a vast volume of logs to identify abnormal behaviors. Moreover, they lack the means to pinpoint issues within AI models for improvement. Hence, in domains such as network intrusion detection, achieving both high accuracy in results and comprehensible explanations for AI algorithms is of paramount importance. Security analysts must understand the decisions made by AI systems and build trust in their capabilities. Therefore, XAI frameworks play a vital role in assisting analysts by providing techniques that elucidate AI decisions, facilitating efficient investigations, and ultimately saving time for the human security analysts.

F. CHALLENGES OF XAI FOR IDS AND NEED FOR EVALUATING XAI

There are several challenges in applying XAI for network intrusion detection. Overall, there is not a black-box XAI technique that can generate explanations that address all six evaluation metrics perfectly (as shown in the results of this work in Section VI and in the work that defined the evaluation metrics [23]). The current challenges can be categorized as shown below:

- **Lack of Transparency:** AI models that are designed for network intrusion detection have a black-box nature. This represents a hard challenge when it comes to interpretation and understanding. This lack of transparency poses a problem in network security because it makes it hard for security analysts to audit and protect systems from possible attacks.
- **Limited XAI Application in Network Security:** Due to unique challenges of network intrusion detection [49], there is still a lack in developing interpretation methods for network intrusion detection systems. This is in contrast to other systems where there are already more XAI methods developed (e.g., text analysis and computer vision). This situation hinders the application of XAI methods in the network security domain.
- **Accuracy and Robustness:** Besides being accurate, XAI methods applied to network security systems need to satisfy extra requirements, such as providing complete and robust explanations. Therefore, by having complete and robust explanations, XAI methods would be more reliable which can ensure their effective application in network security.
- **Evaluation and Comparison:** There is a pressing need for developing evaluation criteria in order to evaluate XAI methods and compare different XAI methods in the network intrusion detection field [50]. Such criteria should cover network security domain requirements (e.g., stability, robustness, reliability, and efficiency), as well as general properties of machine learning models (e.g., accuracy, transparency, and explainability).

IV. FRAMEWORK

The main goal of this work is to provide an XAI evaluation pipeline that derives important metrics for testing each XAI method. Our framework can help in choosing efficient XAI methods that can help security analysts better understand the features of the network traffic and the decisions of the intrusion detection AI models to prevent different types of intrusions on the network under their management. The components of our framework are detailed below.

A. HIGH-LEVEL XAI EVALUATION PIPELINE COMPONENTS

In a high-level fashion, our framework starts by loading well-renowned network intrusion datasets that contain traffic logs from the network of interest. After collecting the network's data, our framework feeds such data into different black-box AI models where these models detect intrusions

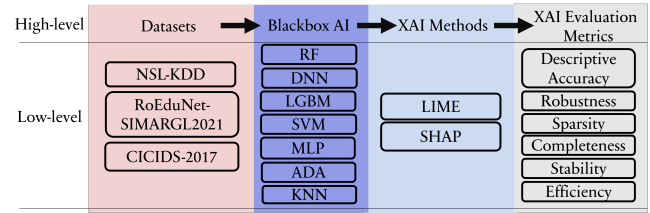


FIGURE 1: An overview of our XAI evaluation framework for network intrusion detection. It considers a diverse set of AI models and network intrusion datasets.

and normal traffic via constructing a multi-class classification problem. In this classification problem, each class represents one possible network intrusion (e.g., denial of service, port-scanning, and brute force attacks). After predicting traffic instances using the black-box AI model, the following step is to pass these predictions into XAI methods to generate the explanations of the instances and the associated predictions. Then, our framework evaluates these XAI methods by leveraging the six XAI evaluation metrics (i.e., descriptive accuracy, sparsity, efficiency, stability, robustness, and completeness).

B. LOW-LEVEL XAI EVALUATION PIPELINE COMPONENTS

We now explain the low-level components of our XAI pipeline. The different components (shown in Figure 1) are explained below.

Network Intrusion Datasets: The first component in our pipeline is loading a network intrusion dataset from the database as a starting point. In our work, we use three popular network intrusion detection datasets which are RoEduNet-SIMARGL2021 [29], CICIDS-2017 [30], and NSL-KDD [21] datasets.

Black-box AI Models: Once the preprocessing of the dataset is complete, we train the black-box AI models where we perform splitting of the data with a split of 70% for the training while leaving the unseen 30% for testing purposes. For this component, we have built seven popular AI classification models including random forest (RF), deep neural network (DNN), adaptive boosting (AdaBoost), multi-layer perceptron (MLP), k-nearest neighbors (KNN), support vector machine (SVM), and LightGBM. For training each model, we choose its specific parameters to ensure the best possible performance (see Section V).

AI Models Selection Criteria: The AI models considered in this work were chosen for the following reasons: The first reason is that they are ubiquitous in many similar works that focus on IDS (e.g., [21], [36], [37]). The second reason is to study the effect of XAI-based feature selection on the performances of AI models in IDS, which enables us to compare our methods with prior works on the three considered datasets [21], [36], [37]. In this manner, we can help keep consistency with the literature in IDS when comparing different models paired with the XAI techniques.

XAI Methods: After constructing the black-box AI models, we use XAI methods to explain such models. For this purpose, we leverage two popular black-box XAI methods which are LIME [25] and SHAP [26]. These methods cover both global and local scopes in evaluating black-box AI models. Furthermore, these techniques are post hoc where they are used after the AI model is trained and its predictions are generated. Thus, these methods are model-agnostic methods and can be used to evaluate diverse sets of black-box AI models. Therefore, LIME and SHAP are well-known XAI techniques for evaluating black-box AI models [23]. We reemphasize that SHAP applies the concept of Shapley value from game theory to score and classify the most important features in a given AI's decision while LIME uses surrogate linear local models to approximate the global AI model and derive explanations on a given sample (here, network traffic instance or sample).

Note that although LIME was originally made for local explanations, we leveraged it to work with the scope of global explanations (as shown in Tables 7, 10, 11, and 13). Our modification can be described as follows. We select how many samples we want to analyze and for each sample, LIME generates a local explanation with the associated scores for each feature. For each feature, we aggregate these scores and store them in a list that is constructed for that feature. For each iteration, we sum the absolute values of those scores for each feature, and at the end, we average all the results by the number of samples used. Finally, we rank the different features where the features with the highest scores would be selected as the most influential ones. We also refer to subsection E.1. for a step-by-step explanation.

Rationale of Choosing SHAP and LIME: The choice of these two frameworks for our pipeline is due to single-feature explainability methods like partial dependence plots (PDP) [51], individual condition expectations plots (ICE) [52], and leave one column out (LOCO) [53] lack the direct ability to capture interactions between various features. This limitation makes it progressively challenging to maintain local model fidelity. Conversely, SHAP and LIME address these constraints of single-feature models effectively. The aforementioned popularity of these two methods for IDS is another reason for focusing on them.

XAI Evaluation Metrics: Our framework mainly seeks to evaluate different XAI methods by generating six evaluation metrics for each XAI method. These six metrics are descriptive accuracy, sparsity, efficiency, stability, robustness, and completeness. First, descriptive accuracy measures the drop in the AI model's accuracy when the top most influential features (generated by XAI methods) are absent. Second, sparsity measures whether the explanation is 'divided' among the features or not (i.e., if many features are influential, then the XAI method is not sparse). Third, stability measures the level of trust in a given explanation. In other words, if the XAI explanation is the same under many trials within the same circumstances, then the XAI technique is stable. Fourth, efficiency is related to the time the XAI method

takes to produce its explanations (e.g., a method that takes too long time to produce explanations is not efficient). Fifth, robustness is how resistant the XAI method is when facing a malicious actor that is trying to attack its integrity (i.e., an attacker might disguise an attack by generating fake (or degraded) yet plausible explanations to avoid detection). Finally, completeness measures the ability of the XAI method to cover every instance. For instance, an XAI method that provides a correct explanation under different corner instances is complete.

C. STEP-BY-STEP ALGORITHMS FOR GENERATING XAI EVALUATION METRICS

We next explain the step-by-step process for each one of our six evaluation metrics (descriptive accuracy, sparsity, efficiency, stability, completeness, and robustness) to generate our XAI evaluation metrics and their results.

C.1. Descriptive Accuracy Step-by-Step

Prerequisite: Feature list in order of importance obtained with SHAP or LIME.

For each one of the seven AI models (Random Forest, DNN, LGBM, MLP, KNN, ADA, SVM) and each one of the three datasets (CICIDS-2017, NSL-KDD, RoEduNet-SIMARGL2021):

- 1) Define $k = (0, 5, 10, 25, 50, 70)$, which is the number of features we are going to subtract from the original features in order of importance (or contribution).
- 2) For each element in k , we obtain the AI model's accuracy. For example, for $k = 0$, we are running a given AI model with all features. For $k = 5$, we are running the same model with all features but minus the top 5 features.
- 3) The k vector becomes the X-axis, and the accuracy of the AI model is the Y-axis.
- 4) We plot the Descriptive Accuracy graph using such axes and analyze the results.

For more information about the Descriptive Accuracy metric, we refer to subsection A in Section VI.

C.2. Sparsity Step-by-Step

Prerequisite: Feature Scores obtained with SHAP or LIME. Note that feature scores are normalized with a min-max technique to be in the range $[0, 1]$.

For each one of the seven AI models (Random Forest, DNN, LGBM, MLP, KNN, ADA, and SVM) and each one of the three datasets (CICIDS-2017, NSL-KDD, and RoEduNet-SIMARGL2021), we apply the Sparsity metric as follows:

- 1) We create an X-Axis with the values $[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]$. These values represent the threshold that we compare with the feature scores.
- 2) For each value of the values in the X-axis (the threshold axis), we apply the formula in Equation (1) below to

determine the sparsity score.

$$Sparsity = \frac{No_Features \text{ s.t. } \{Feature_Score \leq \tau\}}{Total_Number_Features} \quad (1)$$

- 3) Each result generated is a component of the Y-axis representing the Sparsity score for each value in the X-axis. Note that the Y-axis is normalized because its values are divided by the total number of features.
- 4) We use the Y-axis and X-axis to plot the Sparsity graph.
- 5) Area under the curve is generated from plot in step 4.

C.3. Efficiency Step-by-Step

Prerequisite: None necessary.

For each one of the seven AI models (Random Forest, DNN, LGBM, MLP, KNN, ADA, and SVM) and each one of the three datasets (CICIDS-2017, NSL-KDD, and RoEduNet-SIMARGL2021) we apply the Efficiency metric using SHAP or LIME XAI methods as follows:

- 1) After training a given AI model, we applied LIME or SHAP for N samples. For SHAP, we used 1, 100, 500, 2500, and 10000 samples. Meanwhile, for LIME, we used 1 and 1000 samples.
- 2) For each of the runs, we tracked the time needed to complete the XAI evaluation technique.
- 3) We summarize the results from step 2 and analyze which XAI technique paired with which AI model is more efficient.

C.4. Stability Step-by-Step

Prerequisites: None

For each one of the seven AI models (Random Forest, DNN, LGBM, MLP, KNN, ADA, and SVM) and each one of the three datasets (CICIDS-2017, NSL-KDD, and RoEduNet-SIMARGL2021), we apply the Stability metric using SHAP and LIME XAI methods as follows:

- 1) For each dataset, depending on the number of total features, we choose the top- k features to compare in each run. For example, CICIDS-2017 has over 70 features. Therefore, comparing the top $k = 20$ features is a representative amount. In the case of RoEduNet-SIMARGL2021, we evaluated the top $k = 5$ features because this dataset has 29 features in total.
- 2) We check the stability using this formula:

$$Stability = \frac{\bigcap_{i=1}^N X_i}{k},$$

where N is the number of runs, k is the number of top features. In other words, we evaluate the Stability for all the N runs (how many times the top- k features are common in the different N runs). If all the X sets are the same, the stability score will be 1. In the same way, if none of the top features are the same, then the XAI technique is not stable, and the result will be 0.

- 3) We repeat the process for $N = 3$ for each AI model and intrusion detection dataset.

C.5. Robustness Step-by-Step

Prerequisites: The algorithm from the work [24].

For each dataset (CICIDS-2017, NSL-KDD, and RoEduNet-SIMARGL2021), we run the Robustness experiment as follows:

- 1) We train the adversarial model, and the biased model. The objective is to decouple the original explanation from the post hoc XAI explanation (by SHAP or LIME).
- 2) For the adversarial model, we use the top- k (with $k=6$) intrusion features for the unrelated feature. Meanwhile, for the Biased model, we only use the Biased feature.
- 3) We use SHAP and LIME to check if the Biased feature indeed appeared as the top feature for the biased model. Also, for the adversarial model, we check if the unrelated feature and the biased feature appear in the top 3 positions (features).
- 4) To generate the occurrence graph, we run the Robustness metric for 1000 samples. For each run, and each model (adversarial and biased), we stack the top 3 features in a bar graph.
- 5) We then analyze the occurrence of each feature of each model in the text. The biased model is prerogative biased, so the top-1 feature likely is the biased one.
- 6) We look at the adversarial model and check if the unrelated and the biased features are at the top 3 features. The presence of the unrelated feature in rank higher than the biased feature means the explanation was hijacked.
- 7) **Sensitivity graph:** We also use the work of [23] to generate such a sensitivity graph. Considering all datasets, we train OOD classifier models and extract their F1 scores. Such scores are used as the X-axis, then the explanation is run for the adversarial models, and the occurrence of the biased feature appearing as 1st in the SHAP or LIME explanation is tallied and used as the Y-axis.

Discussion: We stress that robustness's main point is showing that such an adversarial attack on XAI methods (explained above) is possible. However, we do not enter the merit of actually performing it in the sense of how to break into a system and switch the correct AI model for the adversarial one used for the explanation to generate the disguised explanation. Therefore, the attacker exploits the fact that the LIME and SHAP are post hoc XAI methods.

C.6. Completeness Step-by-Step

We explain below step-by-step details for both local completeness (on single network traffic sample) and global completeness (on many traffic samples) for completeness XAI evaluation metric used in our work.

C.6.1 Local Completeness (Single Sample) Step-by-Step:

Prerequisites: None

For each one of the three datasets (CICIDS-2017, NSL-KDD, and RoEduNet-SIMARGL2021), we apply the Completeness metric using RF paired with SHAP and LIME XAI methods as follows:

- 1) Split the dataset per class type. In this way, we can evaluate the results per class type.
- 2) Generate an explanation with LIME or SHAP and a prediction of the chosen traffic sample.
- 3) Starting for the top feature in descending order, we perturb the value of such feature in the sample from 0 to 1.0 in steps of 0.1.
- 4) For each perturbation, we generate a new prediction and new explanation with LIME or SHAP.
- 5) If the new prediction is different from the original one, then we stop the program. This is done because if the sample went through a class change, it means its explanation was valid.
- 6) If the new prediction is the same as the original prediction, then we repeat the process for the second most important feature (Steps 4 and 5).

C.6.2 Global Completeness Algorithm

Our proposed algorithm for testing global completeness (for many samples) works as follows:

- 1) Separate the batches by label class
- 2) Generate the explanation for the original sample
- 3) Perturb the top-1 feature and check if the class changes. Then, repeat the process for the following features till the top-k feature. The perturbation will change the value of the feature from 0 to 1 in steps of 0.1.
- 4) If the current perturbed feature fails to change the prediction class, its value is assigned as $|originalvalue - 1|$ in order to keep previous perturbed features with a relevant perturbation value.
- 5) If the class changes, the experiment stops for that sample and a counter registers it.
- 6) Count how many samples changed class when perturbed and this number is divided by the batch size.
- 7) Calculate the percentage number of samples that have valid explanations as follows

$$Completeness(\%) = \frac{\text{No. Samples that Changed Class}}{\text{Total Number of Samples}}$$

D. TOP INTRUSION FEATURES LIST AND USAGE IN EVALUATING XAI

We now show the full list of the top network intrusion features along with their explanations for the three studied datasets since we use those frequently throughout the rest of the paper. In particular, Tables 2-4 show the description for each feature in RoEduNet-SIMARGL2021, CICIDS-2017, and NSL-KDD network intrusion datasets, respectively. We emphasize that we will utilize such features in evaluating the XAI methods on the three datasets in our evaluation (Section V). More specifically, the extraction of top features is absolutely necessary for different XAI evaluation metrics (particularly generating descriptive accuracy, sparsity, and stability) because these metrics analyze these top features and their respective scores directly. Furthermore, these features are used indirectly for the other three XAI evaluation metrics (i.e., efficiency, robustness, and completeness) because

through generating those features we can extract the time taken and whether the explanations are correct or not.

Remark: In our article, Tables 2-4 provide a description of the key features for the RoEduNet-SIMARGL2021, CICIDS-2017, and NSL-KDD datasets, respectively. These tables are intended to highlight some of the key features from each dataset for clarity and context. However, we did indeed utilize all the features listed in Table 5 in our initial experiments. This comprehensive approach allowed us to fully leverage the datasets for our analysis of network intrusion detection. It is important to note that Table 5 summarizes the overall scope of each dataset, including the number of features. The feature selection in our XAI evaluation experiments was tailored to the objectives of those specific analyses. This selective usage of features was based on their importance and relevance to the specific experiment being conducted. We also show the top feature list for all of our three datasets for SHAP and LIME in Tables 27- 28.

E. USAGE OF FEATURES FOR SHAP AND LIME

To obtain a list with the features organized by order of importance (with their respective SHAP or LIME scores), we first have to train each one of the seven AI models for each intrusion detection dataset (RoEduNet-SIMARGL2021, CICIDS-2017, and NSL-KDD). We then apply SHAP Global Summary Plots (or LIME) with all features included. We below detail the step-by-step process for such feature generation process using LIME and SHAP, respectively.

E.1. Feature Generation Step-by-Step for LIME (Global)

- 1) Generate an explanation per network traffic sample.
- 2) Extract the top feature list with its corresponding contribution score from that sample.
- 3) Add the absolute LIME values for that sample to a linked list.
- 4) Repeat the process in steps 1-3 N times for N network traffic samples.
- 5) Divide the feature importance in the final list per the total number of samples.
- 6) Sort the list in descending order to obtain the most influential features for LIME.

We can also generate the feature importance list by default using the SHAP Global Summary Plot [26]. From this plot, we can generate overall feature importance for each AI model, and each intrusion class.

E.2. Feature Generation Steps for SHAP (Global)

- 1) We select how many samples we want to use to generate the global scope and pass it to the SHAP explainer method.
- 2) With the SHAP explainer method, we can generate and extract the SHAP Global Summary Plot and extract the top feature list with the scores from the graph's properties.

V. FOUNDATIONS OF EVALUATION

We next show our detailed evaluation. Our evaluation aims to answer the following questions: (1) What are the main

TABLE 2: Description of main features for RoEduNet-SIMARGL2021 dataset [54].

RoEduNet-SIMARGL2021 Features	Explanation
FLOW_DURATION_MILLISECONDS	Flow duration in milliseconds
PROTOCOL_MAP	IP protocol name (tcp, ipv6, udp, icmp)
TCP_FLAGS	Cumulation of all flow TCP flags
TCP_WIN_MAX_IN	Max TCP Window (src->dst)
TCP_WIN_MAX_OUT	Max TCP Window (dst->src)
TCP_WIN_MIN_IN	Min TCP Window (src->dst)
TCP_WIN_MIN_OUT	Min TCP Window (dst->src)
TCP_WIN_SCALE_IN	TCP Window Scale (src->dst)
TCP_WIN_SCALE_OUT	TCP Window Scale (dst->src)
SRC_TOS	TOS/DSCP (src->dst)
DST_TOS	TOS/DSCP (dst->src)
FIRST_SWITCHED	SysUptime of First Flow Packet
LAST_SWITCHED	SysUptime of Last Flow Packet
TOTAL_FLOWS_EXP	Total number of exported flows

TABLE 3: Description of the main features for the CICIDS-2017 dataset [55].

CICIDS-2017 Features	Explanation
Packet Length Std	Standard deviation length of a packet
Total Length of Bwd Packets	Total size of packet in backward direction
Subflow Bwd Bytes	Average number of bytes in backward sub flow
Destination Port	Destination Port Address
Packet Length Variance	Variance length of a packet
Bwd Packet Length Mean	Mean size of packet in backward direction
Avg Bwd Segment Size	Average size observed in the backward direction
Bwd Packet Length Max	Maximum size of packet in backward direction
Init_Win_Bytes_Backward	Total number of bytes in initial backward window
Total Length of Fwd Packets	Total packets in the forward direction
Subflow Fwd Bytes	Average number of bytes in a forward sub flow
Init_Win_Bytes_Forward	Total number of bytes in initial forward window
Average Packet Size	Average size of packet
Packet Length Mean	Mean length of a packet
Max Packet Length	Maximum length of a packet

features generated by XAI methods for each AI model?; (2) What are the performances of the two XAI techniques under the six XAI evaluation metrics?; and (3) What are the limitations and strengths of black-box XAI methods when applied to network intrusion detection?; and (4) Which black-box XAI method is better under the six metrics?

Before showing our detailed evaluation results, we first detail the experimental setup in this section.

A. DATASET DESCRIPTION

RoEduNet-SIMARGL2021 Dataset [29]: This dataset originates from the SIMARGL project, a collaborative effort supported by the European Union through the Horizon program, in partnership with the Romanian Education Network (RoEduNet). It comprises authentic network traffic data, including features derived from real-time traffic analysis. The dataset is structured following a data schema resembling Netflow [56], a network protocol developed by CISCO for capturing and monitoring network flows.

CICIDS-2017 Dataset [30]: This dataset serves as a benchmark for intrusion detection and was developed by the Canadian Institute for Cybersecurity at the University of Brunswick in 2017. It encompasses six distinct attack profiles, encompassing activities such as brute force, heartbleed, botnet, DoS, portscan, web attack, and infiltration attack. To create a realistic context, the dataset incorporates background traffic generated through a B-Profile system [57],

TABLE 4: Description of main features for the NSL-KDD dataset [21].

NSL-KDD Features	Explanation
duration	Length of the connection
protocol_type	Type of the protocol (e.g., TCP, UDP)
service	Network service on the destination (e.g., HTTP, FTP)
flag	Normal or error status of the connection
src_bytes	Number of data bytes from source to destination
dst_bytes	Number of data bytes from destination to source
logged_in	1 if successfully logged in; 0 otherwise
count	Number of connections to the same host as the current connection in the past two seconds
srv_count	Number of connections to the same service as the current connection in the past two seconds
dst_host_same_srv_rate	Rate of connection to the same service on destination host
dst_host_srv_count	Count of connections to the same service on destination host
dst_host_same_src_port_rate	Rate of connections to same source port on destination
dst_host_serror_rate	Rate of connections that have activation flags indicating various types of errors
dst_host_rerror_rate	Rate of connections that have rejected flags

TABLE 5: Summary and statistics of the three network intrusion datasets used in this work, including the size of the dataset, the number of attack types (labels), and the number of intrusion features).

Dataset	Number of Labels	Number of Features	Number of Samples
CICIDS-2017	7	78	2,775,364
RoEduNet-SIMARGL2021	3	29	31,433,875
NSL-KDD	5	41	148517

which extracts various user behaviors based on network protocols.

NSL-KDD Dataset [21]: This dataset represents an improved version designed to address inherent issues found in the older KDD dataset [58]. It was jointly developed through collaboration between the University of New Brunswick and the National Research Council of Canada. This dataset incorporates a blend of network traffic data, featuring a mixture of various attack types and normal instances. It adheres to a structured data schema, with features extracted from raw traffic data, rendering it a valuable resource for assessing intrusion detection techniques. For our research purposes, we employ the KDDTrain+ subset for model training and the KDDTest+ subset for testing and evaluation [59].

Summary and Statistics of the Datasets: Having explained the three datasets, Table 5 shows the size of each dataset, the number of attack types (labels), and the number of intrusion features.

B. EXPERIMENTAL SETUP

Computing Resources: For the executed experiments, we used a high-performance computer (HPC) that has four NVIDIA A100 GPUs, 64 GPU-accelerated nodes (each with 256 GB of memory), and a single 64-core AMD EPYC 7713 processor (2.0 GHz and 225-watt) reaching a maximum performance of approximately 7 petaFLOPs. This

supercomputer is designed to aid researchers in advanced AI and machine learning tasks [60].

Coding Tools: To make use of several open-source tools and to have our implementation as open-source, we chose the Python programming language along with different black-box AI toolboxes (including Keras and ScikitLearn). We also used other toolboxes (including Pandas and Matplotlib).

XAI Tools: Furthermore, we used the following XAI toolboxes:

(a) **SHAP [61]:** This is a toolkit that allows us to generate feature explanations for our intrusion detection AI models. We used that tool to generate local and global explanations for our AI models.

(b) **LIME [22]:** This is another popular XAI tool that allows generating a surrogate model that is approximated to the original AI model when evaluated for local samples. For this work, we also leveraged LIME to generate global explanations by aggregating its local explanations and averaging the importance of each feature.

AI Models: By pairing seven popular AI classification algorithms (which are deep neural network (DNN) [4], random forest (RF) [12], AdaBoost (ADA) [62], k-nearest neighbor (KNN) [63], support vector machine (SVM) [5], multi-layer perceptron (MLP) [64], and light gradient-boosting machine (LightGBM) [65]), we evaluate black-box AI methods and different components of our proposed XAI framework for network intrusion detection.

Hyperparameters : We provide our main hyperparameter choices for each AI model in Appendix A.

Metrics for Evaluating XAI: To evaluate XAI techniques, we generate the following metrics. First, descriptive accuracy tables show the area under the curve (AUC) for each AI model and XAI technique for each dataset (see Table 7). Second, sparsity tables also show the AUC for each model and the XAI technique for each dataset. However, with a different context for explainability (see Table 10). In particular, lower AUC is favorable for descriptive accuracy while the opposite holds for sparsity. Third, the efficiency timetable for each XAI technique shows the time spent to generate XAI explanations for local (single instance) and global (many instances) scopes (see Table 13). Fourth, there is a stability table that is generated to test trust in XAI explanations. The stability experiment generates top features several times for each XAI method and checks the intersection among these features from different trials with the same conditions (see Tables 11-12). Fifth, for robustness we adapted the code from [24] and generated an adversarial model that decoupled the sample from its explanation and we tested whether we could generate false predictions while providing convincing explanations on them (e.g., an attack on AI model to predict a network intrusion sample as normal while providing convincing XAI explanation on such decision) (see Figures 11-12). Finally, for completeness, we test whether XAI techniques can achieve completeness or not (covering true explanations of all network traffic instances, including

corner cases). For instance, if a perturbation is caused by the top features changing its explanation, but not changing its predicted class, we can not trust such an explanation. Thus, our completeness figures and tables (see Tables 14-16 and Figures 13-15) measure whether the most influential features change the outcomes when perturbed enough.

C. PRELUDE - FEATURE SELECTION

Before presenting detailed evaluation results, we first explain how we obtained the importance scores of intrusion features since such importance will be used during our different experiments. For each model, we generated a SHAP global explanation and a LIME global explanation (shown in Appendix B)².

For descriptive accuracy and sparsity evaluation metrics, a global explanation was generated after running each AI model for each dataset. Then, we stored the list of the explanations and repeated the experiment by removing top-k features ($k = 5$ for RoEduNet-SIMARGL2021, $k = 20$ for CICIDS-2017, and $k = 20$ for NSL-KDD) from the dataset in descriptive accuracy and sparsity cases by using feature's importance value to generate the results. In the case of stability, we generated the explanations similar to the above-described way, but for p different trials (in this study, we have $p = 3$) to evaluate which top-k features intersect ($k = 5$ for RoEduNet-SIMARGL2021, $k = 20$ for CICIDS-2017, and $k = 20$ for NSL-KDD). When evaluating robustness, the feature extraction method is local because the experimental setup consists of evaluating each explanation individually when performing the integrity attack. For completeness, similar to robustness, the feature extraction method is local, but this time these features are used to evaluate the AI model's classification and associated explanation when perturbing the top-k features ($k = 5$ for RoEduNet-SIMARGL2021, $k = 5$ for CICIDS-2017, and $k = 5$ for NSL-KDD). Lastly, the efficiency experiment does not require the extraction of any features. However, we need to generate them to calculate the computational time to run SHAP and LIME in different scenarios.

Having explained the main experimental setups, datasets, and XAI evaluation metrics, we next provide an in-depth metric analysis that has all our evaluation results.

VI. IN-DEPTH METRIC ANALYSIS

A. DESCRIPTIVE ACCURACY

We start by evaluating the descriptive accuracy of XAI methods on our network intrusion datasets. The descriptive accuracy can be interpreted as the importance of intrusion features when taken out. For instance, if a feature is very important in the AI model's decision, then the model's prediction accuracy will be lower if such a feature is taken out. Intuitively, we would expect that the more features you take out, the higher the drop in prediction accuracy. Graphic-wise, we can expect the more features we take

²As mentioned before, we leveraged LIME to aggregate local explanations to obtain a global explanation.

out, the lesser the area under the curve. For the XAI explanation context, the higher the accuracy drop in the model's prediction after removing a feature, the higher the explainability power of such a feature. Figures 2-3 show the descriptive accuracy for SHAP and LIME XAI methods, respectively. Such descriptive accuracy is shown under the three network intrusion datasets considered in this work.

Axis Explanation: The values of features in the X-axis in Figures 2-3 are the number of features removed. Therefore, for zero, it explicitly means that the technique removes zero features (i.e., all intrusion features are used). Likewise, for the value of 40, 40 features are removed, and so on. As a clarification, we remove the features in order of importance, in other words, from the generated feature importance list by SHAP or LIME. Therefore, the higher the value of the X-Axis, the fewer features are used in training and evaluating the model. Other than that, the Y-axis is the accuracy of the AI model analyzed under each feature setup.

Main Insights: As shown in Figures 2-3, our experiments show that SHAP performed better than LIME in terms of global explainability. In particular, most AI models behave as expected when analyzing SHAP curves for descriptive accuracy where the curves tend to slope after features are taken out (i.e., the accuracy decreases as we remove more intrusion features). Furthermore, Table 6 shows which AI models have an accuracy drop when removing top intrusion features for the three datasets (when analyzing Figures 2-3). The ideal expectation is to have an exponential decay in the accuracy, which corresponds to the importance of the extracted features by XAI methods. However, the actual results show that not all models have a drop in accuracy when removing features (see Table 6). Our experiments show that the NSL-KDD dataset has the highest number of AI models with an accuracy drop. Interestingly, the remaining two datasets do not show a consistent continuous accuracy drop for all models.

Overall, SHAP presented the best results when compared to LIME for descriptive accuracy as shown in Table 7. However, we still see a few curves in both LIME and SHAP where the accuracy remains the same or even increases under removing top features (particularly for DNN and MLP models). One possible explanation for such behavior is that all the intrusion features are almost equally contributing from the point of view of these black-box AI models. For neural network models, we believe that the reason for such behavior can be due to the "curse of dimensionality". This happens when we have many more features than predictions which can cause over-fitting or not constructing observations under so many features. In the context of our application, there are latent relationships among the intrusion features which explains why the accuracy remains high even when taking more features out.

We next categorize different AI models according to respective performances of descriptive accuracy metric.

Categorization of AI Models According to Descriptive Accuracy: Since the descriptive accuracy experiment

TABLE 6: A list of AI models that have relevant accuracy drop per dataset and XAI method for the three network intrusion datasets.

	RoEduNet-SIMARGL	CICIDS-2017	NSL-KDD
SHAP	DNN,MLP,ADA	RF,LGBM,SVM	RF,DNN,LGBM,SVM,MLP,ADA,KNN
LIME	DNN,ADA	LGBM,MLP	DNN,LGBM,SVM,MLP,ADA,KNN

TABLE 7: The quantitative results of the descriptive accuracy (i.e., the area under the curve for different AI models under the XAI methods for the three datasets).

	RoEduNet-SIMARGL2021		CICIDS-2017		NSL-KDD	
	LIME	SHAP	LIME	SHAP	LIME	SHAP
RF	0.995	0.975	0.976	0.949	0.845	0.520
DNN	0.527	0.573	0.781	0.769	0.870	0.635
LightGBM	0.997	0.998	0.997	0.992	0.814	0.584
ADA	0.765	0.690	0.817	0.753	0.830	0.596
MLP	0.555	0.575	0.719	0.724	0.823	0.600
SVM	0.909	0.923	0.929	0.821	0.864	0.585
KNN	0.994	0.998	0.997	0.998	0.897	0.668

provided a myriad of results, we divided it into three categories: Perfect, Partially Expected, and Not Expected. We detail below the explanation of these categories, summarized in Table 8 for such detailed results for SHAP and LIME XAI methods considered in our work.

(i) Trends that perfectly align with the expectation: When looking at descriptive accuracy metric, the ideal expected performance is to obtain a descendent curve when removing features. For it to align perfectly with the expected behavior, the value in the Y-axis (accuracy of AI model) has to decrease or be equal to the previous values as the X-axis increases. We refer to the Descriptive Accuracy curves (Figure 2c and Figure 3c) for the MLP AI model for the NSL-KDD dataset for an example of perfect expectation behavior. We also added a list of AI models that attend the expectation in Table 8 for all datasets.

(ii) Trends that partially align with the expectation: For an AI model to align with the expected behavior partially, for the most part, the value in the Y-axis has to decrease or be equal to the last one as the X-axis increases, and the final curve cannot be a flat line. We also list these AI models in Table 8.

(iii) Trends that do not align at all with the expectation: We find some models that have unexpected behavior which includes values in the Y-axis increasing as some values in the X-axis increase. Also, we consider a flat line curve as unexpected behavior. The list of these AI models for the three considered datasets is in Table 8.

AI Models' Behavior Analysis: Since SHAP provided the best performance for this descriptive accuracy metric, we are sticking with it for this analysis across the three datasets and seven AI models. Across the models, KNN was able to keep the accuracy high for CICIDS-2017 and RoEduNet-SIMARGL2021 without visible drops. However, although the NSL-KDD dataset suffers a significant drop in accuracy, KNN remains consistent with its nature of keeping higher accuracy results than the other AI models with fewer features. When looking at SVM, it shines in the RoEduNet-SIMARGL2021 and

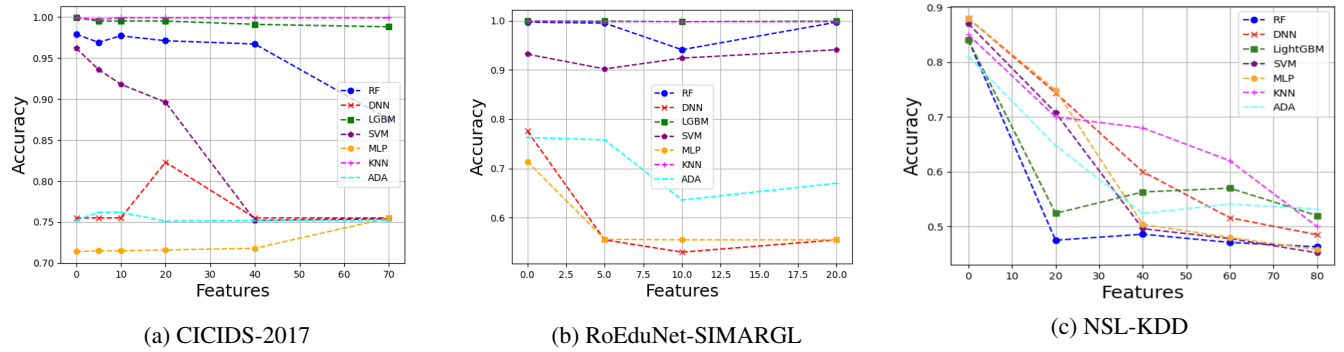


FIGURE 2: Descriptive Accuracy of SHAP XAI method on our three network intrusion datasets. This the graph shows how accuracy declines as key intrusion features are removed, demonstrating SHAP's effectiveness in global explainability.

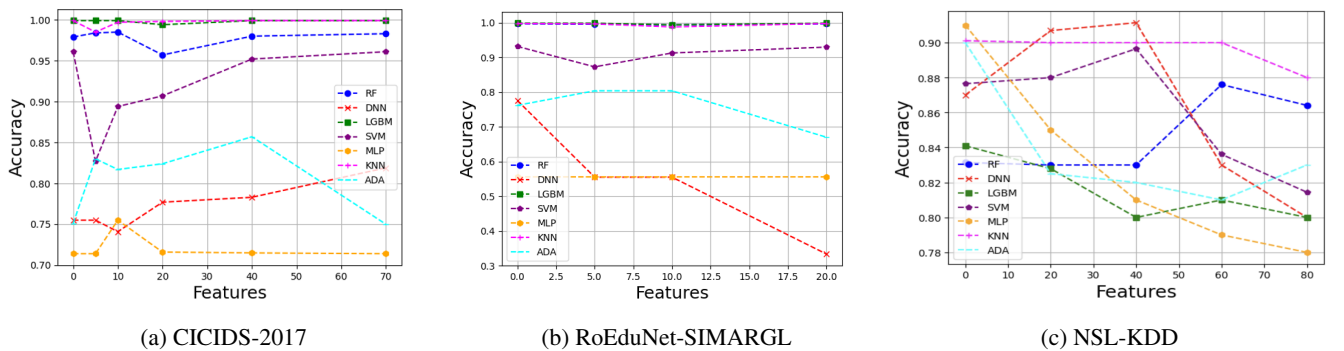


FIGURE 3: Descriptive Accuracy of LIME XAI method on our network intrusion datasets. This the graph shows how accuracy declines as key intrusion features are removed, demonstrating LIME's effectiveness in global explainability.

NSL-KDD datasets, but the accuracy does not drop for CICIDS-2017. DNN and MLP have perfect behaviors for the NSL-KDD dataset. They perform relatively well in RoEduNet-SIMARGL2021, but they have unexpected behavior for CICIDS-2017. Furthermore, ADA performs partially well in RoEduNet-SIMARGL2021 and NSL-KDD datasets, but it lacks performance for CICIDS-2017. LGBM and RF performed partly well for CICIDS-2017 and NSL-KDD datasets. However, they performed poorly for RoEduNet-SIMARGL2021. Interestingly, we could not pinpoint a single model that outperforms the others for all three datasets. Nonetheless, MLP, DNN, and SVM are the ones we recommend experimenting with for descriptive accuracy of intrusion features since they generate better results, compared to other AI models across three datasets.

B. SPARSITY

We then evaluate the sparsity of XAI methods which measures the relevance of feature importance scores (given by the XAI method) for different intrusion features. This is done by comparing how many features are below a certain threshold τ (values from 0 to 1). Here, the threshold is the defined small predefined steps of 0.1 that range from 0 to 1. For example, suppose that the threshold value is given by

$\tau \rightarrow 0$ and there are 19 out of the 20 intrusion features that are below or equal to this threshold. Then, this means that there is only one feature that has an extremely high influence on the AI model's decision, and therefore the sparsity is high. From a graphic-wise point of view, it means that we have more area under the curve for sparsity, as can be seen in Table 10. The intuition here is the higher the area under the curve (AUC), the lower the number of top features needed by the XAI method to provide a good explanation.

Axis Explanation: The values of features in the X-axis in Figures 4-5 are thresholds that increase in steps of 0.1, from 0 to 1. Thresholds are values that we manually set to compare with the LIME or SHAP importance scores to count how many features are below the threshold at a given time. We stress that the higher the importance score, the higher the feature significance from the point of view of the XAI technique. For example, if we analyze a total number of 10 features and 3 out of 10 features have scores less than 0.5, we will have a sparsity score of 0.3 when the threshold is 0.5. Since LIME and SHAP importance scores are normalized using min-max, we guarantee that this metric equals one when the threshold value is one because all features will have at most the importance score of one. We also emphasize the

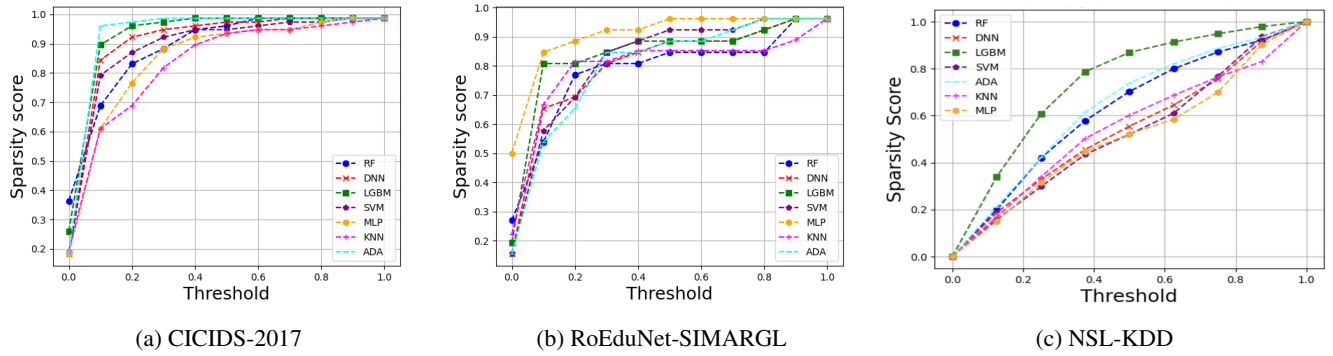


FIGURE 4: Sparsity results for SHAP XAI method for the three datasets. This graph demonstrates the Area Under the Curve (AUC) for SHAP's sparsity across datasets, showing a concentration of explanations in top intrusion features, indicative of higher explainability.

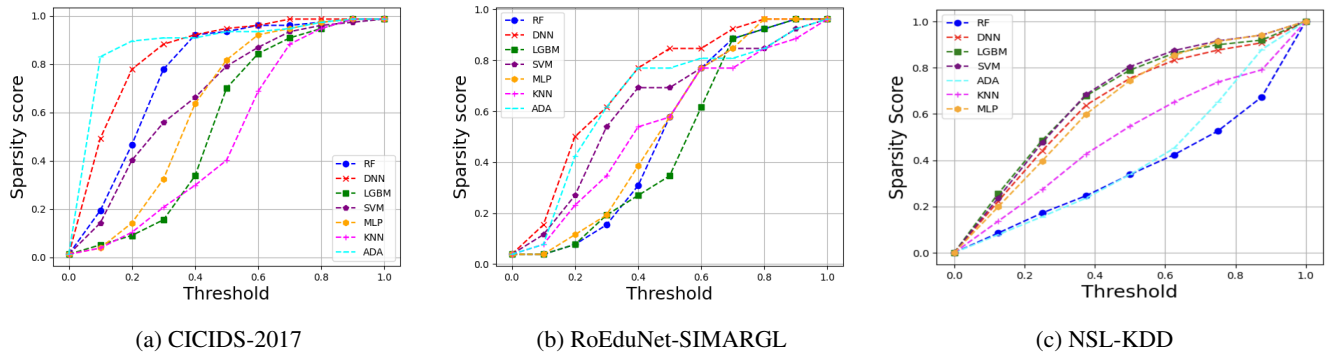


FIGURE 5: Sparsity comparison for LIME XAI method for the three network intrusion datasets. This graph presents LIME's AUC sparsity results, revealing a more linear AUC curve that suggests explanations are spread across a wider range of features, indicating a different pattern of explainability.

TABLE 8: Categorization of AI models' behaviors (behaved as expected, partially expected, and not expected) for the Descriptive Accuracy metric. SHAP outperforms LIME in the second column. Also, SHAP tends to have better results overall, having 13 out of 21 models in the first two columns, while LIME has eight out of 21 models in the first two columns.

Descriptive Accuracy	Perfect	Partially Expected	Not Expected
CICIDS-2017(SHAP)	SVM	RF, LGBM	MLP, DNN, KNN, ADA
RoEduNet-SIMARGL(SHAP)	-	ADA, MLP, DNN	KNN, RF, SVM, LGBM
NSL-KDD(SHAP)	SVM, MLP, DNN	RF, LGBM, KNN, and ADA	-
CICIDS-2017 (LIME)	-	-	LGBM, KNN, SVM, RF, ADA, DNN, MLP
RoEduNet-SIMARGL(LIME)	DNN	ADA	LGBM, MLP, SVM, KNN, RF
NSL-KDD(LIME)	MLP, KNN	ADA, SVM, DNN, LGBM	RF

intuition that a peak on the left-hand side of the graph is the best scenario because it indicates that most features hold low explanation potential. In other words, the explanation potential is dense in a small set of intrusion features.

Conversely, the Y-axis refers to the sparsity score. This value's range is between zero and one, and the number of features that have their importance score below or equal for each threshold value in the X-axis tallies for calculation. Then, the tally is divided by the total number of features.

Results Criteria: Here, we try to shed more light on the sparsity results. The expected Sparsity curve is the one that

peaks on the left-hand side of the graph. If it draws an almost inverse "L" shape, it indicates that a small set of features holds the most potential for explanations. Please see the ADA SHAP curve for CICIDS-2017 in Figure 4 for the best sparsity result obtained in this work.

Main Insights: Figures 4-5 show the sparsity for SHAP and LIME XAI methods, respectively. In these figures, sparsity is shown under all three network intrusion datasets. Again, SHAP shows superior performance in terms of sparsity compared to LIME. In particular, the curves that represent different AI models offer a higher area under the

TABLE 9: Categorization of AI models for their performance in Sparsity metric. They are classified as Perfect Performance, Partially Good Performance, and Poor Performance. Please note that SHAP outperforms LIME with 15 out of 21 models in the first two columns, while LIME has 7 out of 21 models in the first two columns.

Sparsity	Great Performance	Partially Good Performance	Poor Performance
CICIDS-2017(SHAP)	ADA, LGBM, DNN, RF, SVM	MLP, KNN	-
RoEduNet-SIMARGL(SHAP)	MLP, LGBM, KNN	RF, DNN, SVM, ADA	-
NSL-KDD(SHAP)	LGBM	-	MLP, SVM, ADA, RF, KNN, DNN
CICIDS-2017 (LIME)	ADA, DNN	RF	MLP, KNN, SVM, LGBM
RoEduNet-SIMARGL(LIME)	-	ADA, DNN	MLP, KNN, SVM, LGBM, RF
NSL-KDD(LIME)	-	LGBM, SVM	MLP, KNN, DNN, RF, ADA

curve compared to LIME (i.e. SHAP's curves spike towards 1 with smaller thresholds). This means that XAI explanations are more concentrated in fewer intrusion features, therefore SHAP has low sparsity. Overall, Figures 4-5 show that SHAP has the best results for CICIDS-2017, RoEduNet-SIMARGL, and NSL-KDD datasets. In these results, we have an exponential growth shape showing that the explanation is concentrated only in a few top intrusion features. This is indicative of higher explainability potential for these datasets. The remaining results (i.e., NSL-KDD using LIME and SHAP, and CICIDS-2017 and RoEduNet-SIMARGL for LIME) have a few models that have AUC curves with a linear relationship indicating that the explanation is spread out through more features. From a quantitative point of view, Table 10 shows the area under the curve for Figures 4-5 for the three datasets and the two XAI methods.

Categorization of AI Models According to Sparsity: We also broke down the results into three categories in Table 9 according to Equation (1). The categories are Perfect, Partially Good Performance, and Poor Performance. The criteria is to check the curve when the threshold is 0.2 on the X-axis. If the Y values are greater or equal to 0.8, we classify the curve as a Perfect performance. If the values are between 0.4 and 0.8, we classify the curve as a Partially Good Performance. Otherwise, we classify the curves as Poor Performance.

Sparsity Categories Discussion: The results in Table 9 show that SHAP performed better than LIME for Sparsity metric, which indicates that SHAP is attributing the importance scores more correctly compared to LIME. Thus, SHAP picks (or identifies) overall better features than LIME. Although the relationship between Descriptive Accuracy and Sparsity is not causal, there exists a correlation between the two. Overall, it is no surprise that SHAP outperformed LIME in Descriptive Accuracy since the results for Sparsity foreshadow it. However, one can note that some models that achieved Perfect Performance in Table 9 for Sparsity did not perform well in Table 8 for Descriptive Accuracy. Based on these findings, we can conclude that this relationship between the two metrics is not a direct cause-effect relationship for our network intrusion detection task.

C. STABILITY

We next evaluate the stability of different XAI methods. The stability experiment was generated by running each AI model

TABLE 10: The quantitative results for sparsity metric. We show the area under the curve (AUC) for different AI models for both XAI methods and three datasets.

	RoEduNet-SIMARGL2021		CICIDS-2017		NSL-KDD	
	LIME	SHAP	LIME	SHAP	LIME	SHAP
RF	0.519	0.788	0.768	0.893	0.370	0.623
DNN	0.707	0.809	0.844	0.916	0.646	0.542
LGBM	0.480	0.846	0.552	0.937	0.673	0.742
ADA	0.653	0.805	0.882	0.942	0.411	0.640
MLP	0.534	0.911	0.629	0.855	0.643	0.514
SVM	0.619	0.825	0.679	0.896	0.678	0.527
KNN	0.553	0.803	0.505	0.837	0.507	0.548

and associated XAI explanation three times. We then measure the stability by calculating the percentage of features that intersect in the three runs among the total number of features. The stability range is from zero to one. An XAI method that has stability closer to one is more stable and reliable (i.e., does not change explanations in different identical runs). We measure such stability both in the global (on many traffic instances) and local (on single traffic instances) scopes.

Main insights: As expected, local stability is higher than global due to the fact that it runs the same instance instead of a collection of samples, which are prone to have more differences by pure chance. From our experimental results (see Tables 11-12), we notice that LIME and SHAP perform roughly the same with respect to this category. For the CICIDS-2017 dataset, global LIME has a better performance compared to SHAP for four models (RF, DNN, LGBM, and MLP) while SHAP performed better for only two models (SVM and KNN). For CICIDS-2017, local LIME performed better in two models (RF and LGBM) and it was tied with SHAP for the other five models. For the RoEduNet-SIMARGL2021 dataset, global LIME outperforms SHAP one time (LGBM) while the same is true for SHAP (DNN), and the remaining results are tied. For RoEduNet-SIMARGL2021, local SHAP outperforms LIME one time (DNN) and the remaining results are tied. For the NSL-KDD dataset, there is a noticeable difference in the stability of LIME and SHAP explanations across different AI models. In terms of global stability, LIME showed better stability with RF, DNN, and MLP, while SHAP was more stable with LGBM, ADA, SVM, and KNN. However, the local stability of LIME and SHAP varied with different models, indicating a nuanced performance that depends heavily on the specific combination of the dataset, AI model, and XAI explanation method.

TABLE 11: The global stability evaluation metric of XAI methods. We measure such stability by calculating the percentage of features that intersect in different runs among the total number of features for several traffic instances.

Dataset (XAI Method)	RF	DNN	LGBM	ADA	MLP	SVM	KNN
CICIDS-2017 (LIME)	1.00	1.00	1.00	1.00	1.00	0.95	0.99
CICIDS-2017 (SHAP)	0.35	0.50	0.65	1.00	0.28	1.00	1.00
RoEduNet-SIMARGL2021 (LIME)	1.00	0.20	1.00	1.00	1.00	1.00	1.00
RoEduNet-SIMARGL2021 (SHAP)	1.00	0.40	0.60	1.00	1.00	1.00	1.00
NSL-KDD (LIME)	1.00	0.95	0.90	0.90	0.95	0.65	0.80
NSL-KDD (SHAP)	1.00	0.95	1.00	0.95	1.00	1.00	0.85

TABLE 12: The local stability evaluation metric of XAI methods. We measure such stability by calculating the percentage of features that intersect in different runs among the total a number of features for a single traffic instance.

Stability	RF	DNN	LGBM	ADA	MLP	SVM	KNN
CICIDS-2017 (LIME)	1.00	1.00	1.00	1.00	1.00	1.00	1.00
CICIDS-2017 (SHAP)	0.35	1.00	0.40	1.00	1.00	1.00	1.00
RoEduNet-SIMARGL2021 (LIME)	1.00	0.40	1.00	1.00	1.00	1.00	1.00
RoEduNet-SIMARGL2021 (SHAP)	1.00	1.00	1.00	1.00	1.00	1.00	1.00
NSL-KDD (LIME)	0.95	0.95	0.80	0.95	1.00	0.83	0.85
NSL-KDD (SHAP)	1.00	0.80	1.00	0.90	0.70	0.80	0.70

D. EFFICIENCY

We next evaluate the efficiency of different XAI methods. Recall that the efficiency experiment does not require the extraction of any features, but it generates them to evaluate the computational time it takes to run SHAP and LIME in different scenarios. Table 13 shows such efficiency results for SHAP and LIME XAI methods for the three network intrusion datasets. We also show the efficiency from global and local scopes. Globally, one of the main differences between LIME and SHAP when looking at the efficiency metric (shown in Table 13) is the consistency of LIME for all AI models. In all models but KNN, LIME has roughly the same computational time. On the other hand, while SHAP performs better for LGBM, RF, and DNN models, SHAP's efficiency drops for SVM and MLP and it plummets for ADA and KNN. Thus, it is non-practical to use MLP and KNN for real-time IDS. In the local scope, Table 13 shows that SHAP is more efficient than LIME for all of the seven AI models.

Given these results, we have the following recommendations from a computational efficiency perspective: (i) the best choice for SHAP is to pair it with LGBM, RF, or DNN for global explanations, (ii) the usage of SHAP is better than LIME for local explanations, and (iii) adapting LIME for global explanations is more efficient compared to SHAP.

E. ROBUSTNESS

To test the robustness of XAI methods, we are adapting the method proposed in [24] for our network intrusion detection application domain. This method consists of generating an adversarial model and a biased model to decouple the prediction from the post hoc XAI explanation. For each dataset (RoEduNet-SIMARGL2021, CICIDS-2017, and NSL-KDD), we use the top intrusion features for training such models. As an example, to analyze a denial of service (DoS) sample for the CICIDS-2017 dataset, we use only the flow duration feature for constructing the extremely biased

model (as shown in Figure 6a). In this example, we also create an unrelated fake feature that correlates with the biased model prediction for the adversarial model (as shown in Figure 6b). The flow duration feature was chosen because it is one of the main flags for a DoS attack. Hence, if this experiment is successful it means that an attacker can deploy a DoS attack and trick the security analyst by showing him a fabricated convincing explanation for normal traffic although it is a DoS attack. Thus, the attacker can hide and compromise the XAI explanation integrity. We also conduct a similar experiment for the NSL-KDD dataset focusing on the `dst_host_same_srv_rate` feature which is identified as the most influential attribute in this dataset [24].

We first show the percentage of samples for which each feature appears in the top 3 features (according to LIME and SHAP rankings of feature importance) for both biased and adversarial classifiers. Figure 8 shows this percentage for the CICIDS-2017 dataset. Note that the biased model (Figure 8a) always has the biased feature as the top feature as expected. Meanwhile, the adversarial model (Figure 8b) shows that the unrelated feature is the most important one while the biased feature has only a few occurrences as the second or third feature. Similarly, Figure 9 and Figure 10 show the results of the above experiment for RoEduNet-SIMARGL2021 and NSL-KDD datasets, respectively.

Bar Plots: Comparing the bar plots generated for SHAP for the three datasets, we see that the biased feature dominates the first place in all occurrences (see Figures 8a, 9a, and 10a). The difference lies in the second and third feature columns for the adversarial model in Figures 8b, 9b, and 10b. For all three datasets, although these figures show that the unrelated feature massively appears as the top feature, there is a variation in how much the biased feature appears in the second or third column. Since this is the feature the attacker is trying to hide, the less this biased feature arises, the less robust the XAI for that dataset. In this context, for SHAP, CICIDS-2017 and RoEduNet-SIMARGL2021 have roughly the same performance while NSL-KDD shows higher vulnerability since the biased feature only appears as the third feature. Similarly, the same findings applies to LIME in Figures 8c, 9c, 10c, 8d, 9d, and 10d. However, LIME performs better than SHAP for the NSL-KDD dataset, bringing more exposure to the biased feature. On the other hand, LIME performs worse than SHAP for CICIDS-2017 and RoEduNet-SIMARGL2021 datasets with fewer occurrences of the biased feature. Overall, the SHAP technique performed better than LIME for two of the three datasets selected for this study in this experiment.

A crucial concept for understanding our robustness experiment is the out-of-distribution (OOD). As defined by the paper [24], OOD instances are generated via perturbation and do not belong to the original evaluated datasets. These instances are used in the experiment to detect biases in the dataset. Figures 11 and 12 show the relationship between these perturbed samples and dataset samples. When an OOD classifier fails to differentiate between them, it correlates

TABLE 13: The efficiency (amount of time in hours) for generating SHAP and LIME explanations for different AI models and different numbers of samples. Note that LIME is stable across all models, but KNN. On the other hand, SHAP has better performance with RF, DNN, and LGBM.

SHAP Efficiency (Samples x Hours)	RF	DNN	LGBM	MLP	ADA	SVM	KNN
100	0.001	0.001	0.001	0.014	0.129	0.041	1.104
500	0.002	0.004	0.001	0.925	3.557	0.841	28.33
2500	0.011	0.029	0.002	21.18	99.00	20.87	706.0
10000	0.046	0.306	0.006	385.0	1512	343.0	1654
LIME Efficiency (Samples x Hours)	RF	DNN	LGBM	MLP	ADA	SVM	KNN
1000 (RoEduNet-SIMARGL2021)	1.401	1.452	1.423	1.466	1.416	1.483	0.333
1000 (CICIDS-2017)	3.812	3.766	3.750	3.855	3.956	3.816	15.81
1000 (NSL-KDD)	2.287	2.110	2.337	2.389	2.478	3.112	12.87
SHAP Local Efficiency (Minutes)	0.001	0.001	0.001	0.001	0.001	0.001	0.001
LIME Local Efficiency (Minutes)	0.351	0.311	0.294	0.284	0.294	0.306	0.357

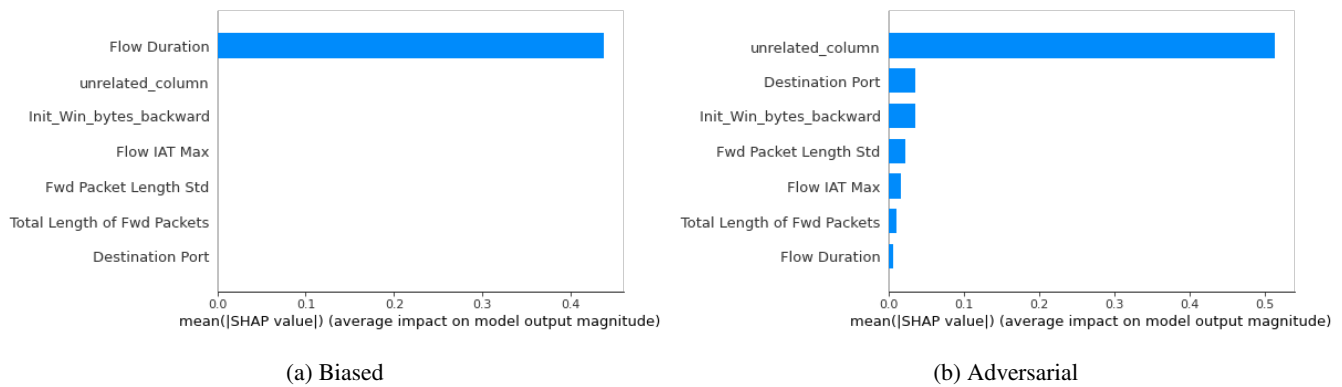


FIGURE 6: An example of DoS sample from CICIDS-2017 dataset for robustness experiment using SHAP. In (a), the feature list (with flow duration as the top feature) under a biased explanation is shown. In (b), the list (with the engineered feature as the top feature) after the adversarial model's classification is shown.

to high vulnerability. Considering that this attack is based on generating synthetically random noise perturbations to deceive XAI methods (here SHAP and LIME), it can become potentially hard to identify whether a sample is an OOD or a generated noise. To identify such differences, we train different OOD classifiers where the efficiency of each classifier is measured by the number of times in which flow duration is the most important feature in the XAI explanation. If LIME and SHAP detect flow duration as the most important feature while the OOD classifier is not able to differentiate whether the sample is an outlier or a perturbation, then the adversarial attack did not succeed. We use the F-1 score metric to evaluate the performance of the OOD classifier [24].

Sensitivity Graphs: We evaluate our robustness experiment with a sensitivity graph where Figures 11 and 12 display the attack's effectiveness for LIME and SHAP, respectively. They show that when our biased feature (i.e., flow duration for CICIDS-2017 and RoEduNet-SIMARGL2021, and dst_host_same_srv_rate for NSL-KDD) is the most relevant, the F-1 score of the OOD classifier is low. Figures 11-12 show that LIME is more robust than SHAP because it is harder to be deceived until a high threshold given by the F-1 score (Figure 11). To the right-hand side of this F-1 threshold (i.e., roughly higher

than 0.8), LIME can be deceived in these cases. In contrast, Figure 12 depicts that SHAP has a more gradual behavior where it has an initial interval (with an F-1 score of [0, 0.3]) that is hard to deceive, however after that threshold almost all results can be swindled. After this interval, the behavior is gradual and the higher the F-1 score the higher the likelihood of SHAP being deceived by the attack. Overall, although LIME and SHAP are both vulnerable to such attacks starting at a certain threshold, our experiments show that LIME is slightly more robust since its initial threshold towards attacks of this kind is higher compared to SHAP.

F. COMPLETENESS

Recall that an XAI technique is considered complete if it can generate true explanations for every sample, even corner cases and outliers. This notation motivates our experiments in order to check the completeness of SHAP and LIME XAI methods. Our experiments can be described by the following steps: (i) Generate an explanation of each network traffic sample and check its explanation, (ii) Perturb the top-k features with values that range from 0 to 1 with steps of 0.1, and (iii) Check the classification decision on the perturbed sample. If the perturbations are not able to change the predicted class of the network traffic sample, it means that the original

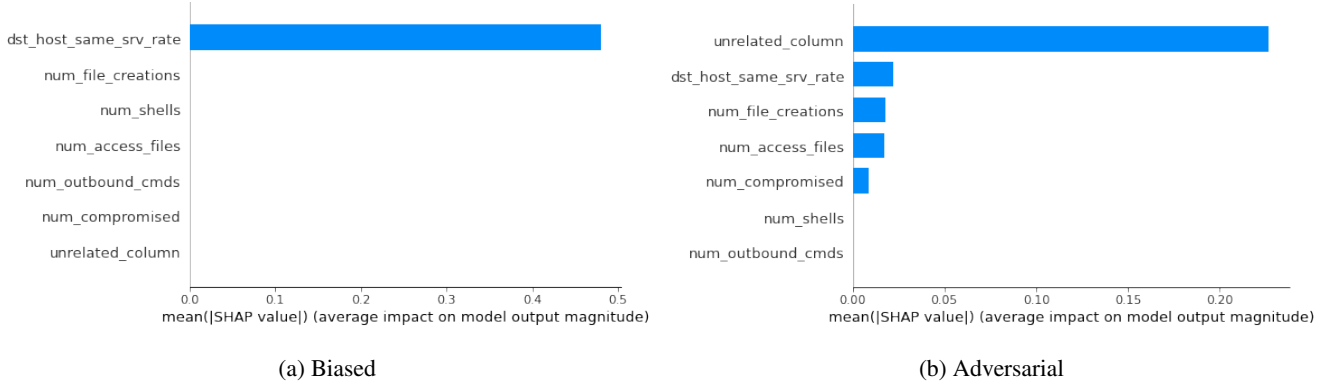


FIGURE 7: An example of DoS sample from NSL-KDD dataset for robustness experiment using SHAP. In (a), the feature list (with flow duration as the top feature) under a biased explanation is shown. In (b), the list (with the engineered feature as the top feature) after the adversarial model's classification is shown.

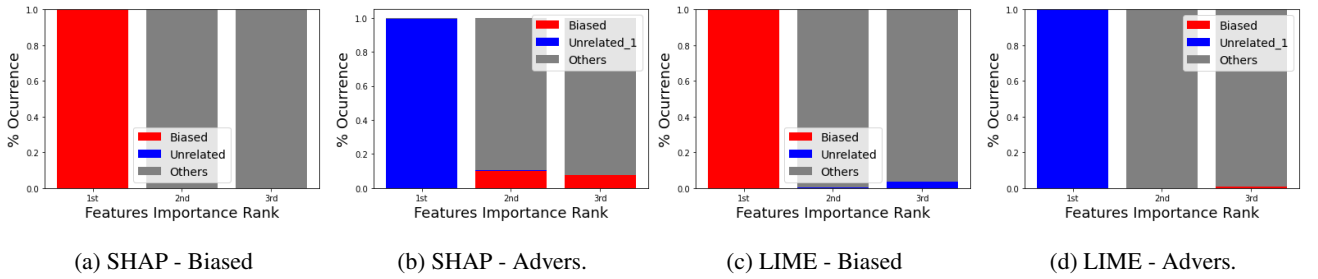


FIGURE 8: The percentage of data samples in the CICIDS-2017 dataset for which biased and unrelated features appear in top-3 features (according to LIME and SHAP rankings of feature importance) for the biased classifier (in (a) and (c)) and adversarial classifier (in (b) and (d)) that uses one uncorrelated feature.

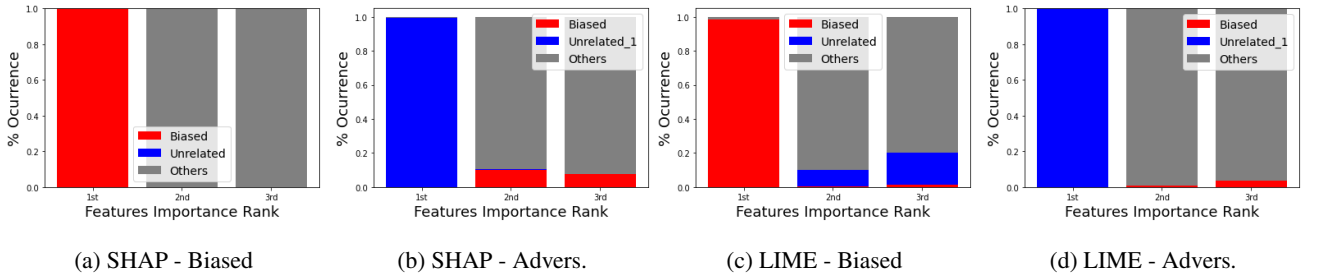


FIGURE 9: The percentage of data samples in the RoEduNet-SIMARGL2021 dataset for which biased and unrelated features appear in top-3 features (according to LIME and SHAP rankings of feature importance) for the biased classifier (in (a) and (c)) and adversarial classifier (in (b) and (d)) that uses one uncorrelated feature.

explanation is not relevant or degenerated. Therefore, the XAI technique is not complete because it is not able to explain that singular sample. On the other hand, if the prediction on the perturbed sample changes when compared to the original, we can say that the explanation for that sample is complete. We performed such analysis on a batch of 1000 samples from each class to test completeness for each class.

We next show evaluation results for both global completeness and local completeness.

Local Completeness: We first clarify our completeness experiments via an example from the RoEduNet-SIMARGL2021 dataset. We extract a network

traffic sample that we know its ground-truth label which is a DoS. We first generate the XAI explanation for this original DoS sample (see Figure 13a for SHAP). The original explanations show that TCP_WIN_SCALE_IN is the top-1 feature. Thus, we perturbed this top intrusion feature and changed its value to 0. We next show the explanation under 1 perturbation (of the top feature) in Figure 13b. It shows that the predicted class explanation has not changed except that the TCP_WIN_SCALE_IN becomes the second top feature. Since this is a positive indication of a valid XAI explanation, we perturbed the second top feature TCP_WIN_MAX_IN, and show the explanation under two perturbations in

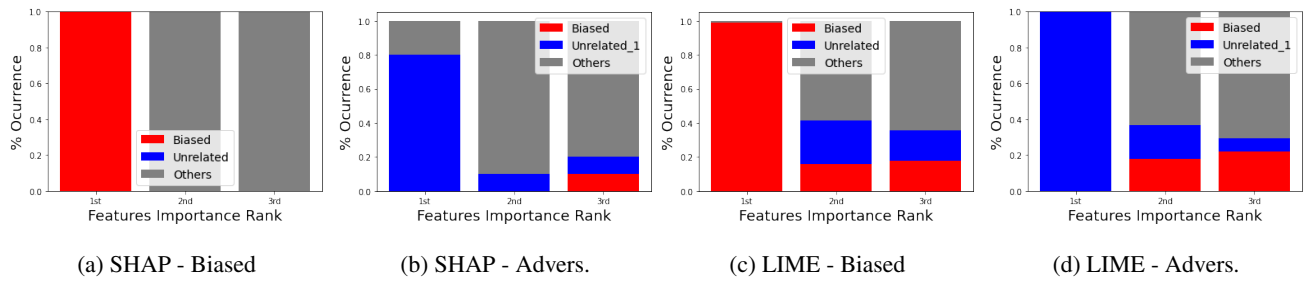


FIGURE 10: The feature occurrence percentage of data samples in the NSL-KDD dataset for which biased and unrelated features appear in top-3 features (according to LIME and SHAP rankings of feature importance) for the biased classifier (in (a) and (c)) and adversarial classifier (in (b) and (d)).

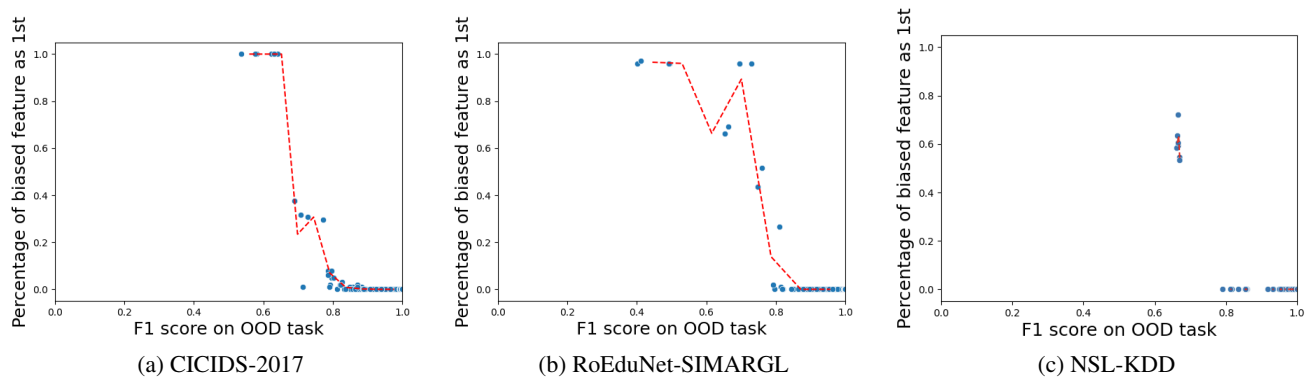


FIGURE 11: Robustness Sensitivity of LIME to OOD attacks for all three network intrusion datasets. This graph displays the robustness of the LIME method for network intrusion datasets, using the F-1 score to evaluate its sensitivity to synthetically generated noise perturbations. It highlights the threshold beyond which LIME becomes more susceptible to being deceived by OOD attacks.

Figure 13c. After the second perturbation, we observe that the class has changed which indicates that the XAI method (SHAP) generated a correct explanation for this sample. Figure 14 shows a similar trend for LIME. We also show another local completeness example using a Probe attack sample from the NSL-KDD dataset. Such an example is shown in Figure 15. It shows that after the 1 perturbation, the top intrusion feature for NSL-KDD moved to third position and after 2 perturbations it stayed in the same third position.

To further test the local completeness, we examined a setup consisting of 100 samples from each intrusion class, and each of those samples receives individual perturbations for each one of its features until a change of intrusion class occurs. Figure 16 shows the results of this experiment which shows the amount of perturbation needed to change the intrusion class of a network traffic sample. In this figure, the y-axis represents the remaining samples of unchanged samples, while the x-axis represents the amount of perturbation applied to the sample. As the perturbation increases, the probability of a class change also increases. In this experiment, LIME and SHAP present similar behavior for CICIDS-2017, RoEduNet-SIMARGL2021, and NSL-KDD. Figure 16 shows that intrusion classes in CICIDS-2017 have a better explanation than RoEduNet-SIMARGL2021

and NSL-KDD, given that it requires less perturbation to change intrusion class. For some scenarios, the curves did not converge to zero because the amount of perturbation applied was insufficient to produce a change. This is consistent with our findings in Tables 14-16. However, since a specific perturbation exists to induce an intrusion class change, we can find via trial and error how much to perturb the intrusion features to change the class.

Global Completeness: Since the above analysis was done locally, the main question that may arise here is: *is this completeness true for all samples considering all intrusion classes?* To answer this question, we generalize the experiment to run a batch of each class (with the size of 1000 samples for each class). The number of maximum features to be perturbed was chosen to be the top 5 features. Tables 14-16 show our global completeness experiment for all of our three datasets for both SHAP and LIME. Overall, the results in Tables 14-16 show that SHAP and LIME are not complete because they cannot generate complete explanations for most intrusion classes. This means that SHAP and LIME were not able to change predicted class even after generating substantial perturbations in the five top features. On the other hand, its explanation changes every time a perturbation happens. If the original top five features were truly relevant,

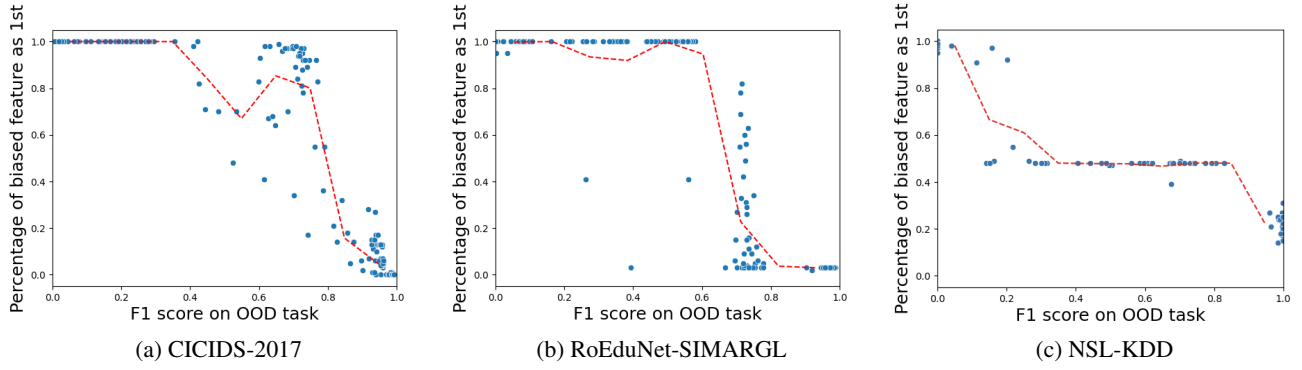


FIGURE 12: Robustness Sensitivity of SHAP to OOD Attacks for all three network intrusion datasets. This figure illustrates SHAP's robustness against OOD attacks across the same datasets, showing a gradual shift in its vulnerability as indicated by the changing F-1 scores. The graph pinpoints the F-1 score threshold where SHAP's resistance to deception (adversarial attack) begins to weaken.

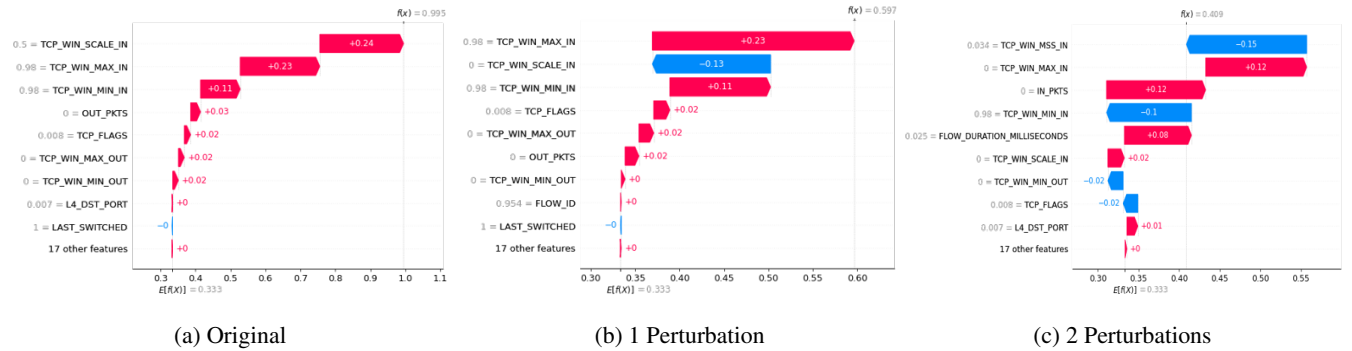


FIGURE 13: An example of completeness experiment on DoS sample from RoEduNet-SIMARGL2021 dataset. This experiment uses using SHAP method. In (a), we show the original XAI explanation. In (b), we show it after the top feature is perturbed. In (c), we show it after the second top feature is perturbed.

it should have a great impact on the model's prediction which did not happen. Instead, new explanations are generated each time. For the RoEduNet-SIMARGL2021 dataset, Table 14 shows that this incompleteness is particularly observed for normal samples. This can be explained because there are many features that can explain 'Normal' traffic. When those original features are perturbed, SHAP can rely on other features to generate its explanation. Table 14 also shows that both SHAP and LIME have valid explanations for all attack samples analyzed. Table 15 displays the results for the CICIDS-2017 dataset. In contrast to Table 14 (in which SHAP and LIME achieve similar performance), Table 15 shows that SHAP performed better than LIME on CICIDS-2017 dataset with more intrusion classes. Since we use the same method for feature perturbation for SHAP and LIME, we conclude that LIME tends to under-perform in terms of global completeness when dealing with multiclass classification with more intrusion labels. For the NSL-KDD dataset, Table 16 shows that the performance of SHAP and LIME differs significantly for this dataset. In every class, SHAP performs noticeably better than LIME, especially in the "Normal" and "DoS" categories. This discrepancy confirms

SHAP's effectiveness for complex and diverse datasets like NSL-KDD. To sum up, our experiments highlight that SHAP is more complete than LIME, indicating that it can provide more thorough and dependable interpretability in intricate classification scenarios.

Common Features across XAI Methods: We provide a heatmap that shows the common top features across different AI models and XAI methods for all of the three datasets. Figure 17 shows such heatmaps for all three datasets. It shows that there is very little overlap between the top features found by SHAP and LIME. Perturbing these features, we found that the top features of SHAP cause a discernible class transition, highlighting their importance. On the other hand, perturbing LIME's top features does not result in a noteworthy variation, suggesting that LIME may be less significant in the classification context. This difference is further supported by the global completeness in Tables 15-16, which shows that higher shift in resulting intrusion class in response to perturbations in SHAP's features than in LIME's, indicating that SHAP is more reliable at identifying pertinent intrusion features.

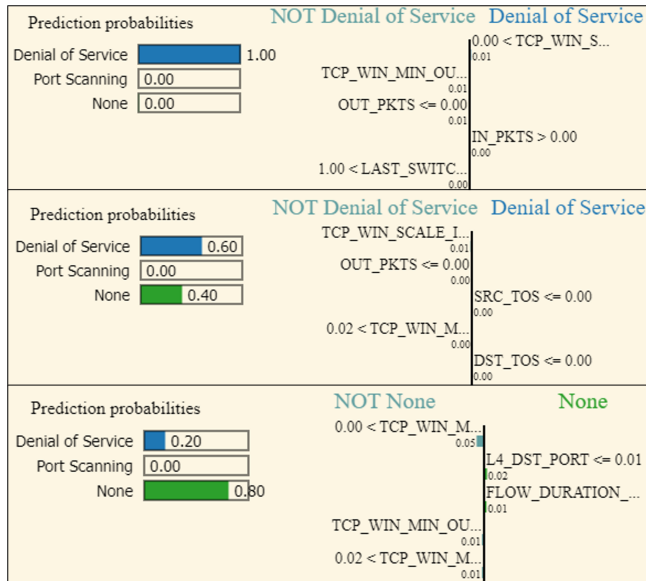


FIGURE 14: The LIME analysis of completeness experiment on the DoS sample from RoEduNet-SIMARGL2021 dataset. In the first quadrant, we show the original explanation. In the second quadrant, we show it after the top feature is perturbed. In the third quadrant, we show it after the second top feature is perturbed.

TABLE 14: The percentage of samples that are complete (i.e., with valid explanations) for each intrusion class using SHAP and LIME for the RoEduNet-SIMARGL2021 dataset.

	Normal	DoS	Port Scan
SHAP	31.1%	100%	100%
LIME	26%	100%	100%

G. STATISTICAL ANALYSIS

We believe that a statistical analysis would bring the validation needed to support our findings. Hence, we chose to apply the non-parametric Wilcoxon signed-rank statistical test to the three datasets used in this work. The rationale behind our choice is that our datasets do not have a normal distribution. First, we prepare seven models (i.e., this will generate 21 p -values for each pair combination of models). Then, we apply the significance test using k-fold cross-validation and their accuracy scores for each model. The result will generate two samples, one for each model (for the two models being compared against each other). Later, we applied the Wilcoxon test to obtain the difference between both samples. If they differ from each other in a significant matter, one can imply that there is significant evidence that one model is more accurate than the other. The algorithm ranks which differences have the highest absolute difference value. Finally, we check whether the calculated p -values are lower than 0.05 to reject the null hypothesis (i.e., there is no difference in the performance of the two models being compared). We have added our statistical analysis results (which are shown in Table 17).

TABLE 15: The percentage of samples that are complete (i.e., with valid explanations) for each intrusion class using CICIDS-2017 dataset.

	Normal	DoS	Port Scan	Bot
SHAP	92.9%	61.4%	50 %	99%
LIME	16%	2%	0%	28%
	Brute Force	Infiltration	Web Attack	
SHAP	100%	50 %	18.4%	
LIME	0%	0 %	8%	

TABLE 16: The percentage of samples that are complete for each class using the NSL-KDD dataset.

	Normal	DoS	Probe	U2R	R2L
SHAP	28.4%	85.3%	63.4%	59.2%	76%
LIME	3%	8%	6%	11%	3%

Furthermore, considering the three datasets, we observe that most pair models reject the null hypothesis ($p < 0.05$). Therefore, there is evidence that one model performs better than the other in most cases. Thus, we can separate the tests into two groups: The group with statistical significance (i.e., $p < 0.05$). And the group that does not have statistical significance ($p > 0.05$). We can further divide the first group into two subgroups by analyzing the medians of their array of accuracy scores of the estimator for each run of the cross-validation method. The first subgroup, indeed, has a model that is performing better (i.e., one model has a higher median than the other one). For this case, we highlighted the best model in bold in Table 17. However, for the second subgroup (i.e., both models display the same median), we cannot know which is better, but only that they are statistically different enough due to the evidence shown by the Wilcoxon test. In this case, we highlighted both models in bold in Table 17.

Summary of Results: Table 18 summarizes the results for both black-box XAI techniques (SHAP and LIME) with respect to the six XAI evaluation metrics considered in this work. Overall, SHAP achieved the best results throughout the experiments performed in this work using the three datasets for network intrusion detection (i.e., CICIDS-2017, RoEduNet-SIMARGL2021, and NSL-KDD). In particular, SHAP outperformed LIME in completeness by generating relevant explanations, especially for CICIDS-2017 and NSL-KDD. The hypothesized reason is due to having more intrusion detection types than RoEduNet-SIMARGL2021 in these datasets. Also, SHAP was able to select more relevant features than LIME for descriptive accuracy and sparsity metrics. Moreover, for stability and efficiency, the results were model-dependent. For Robustness, LIME performed better for NSL-KDD while SHAP performed better for CICIDS-2017 and RoEduNet-SIMARGL2021.

H. OVERALL DISCUSSION OF RESULTS

We believe that our evaluation results show that our work can be a stepping-stone towards evaluating XAI frameworks for real-world IDS. We evaluated six evaluation (that cover

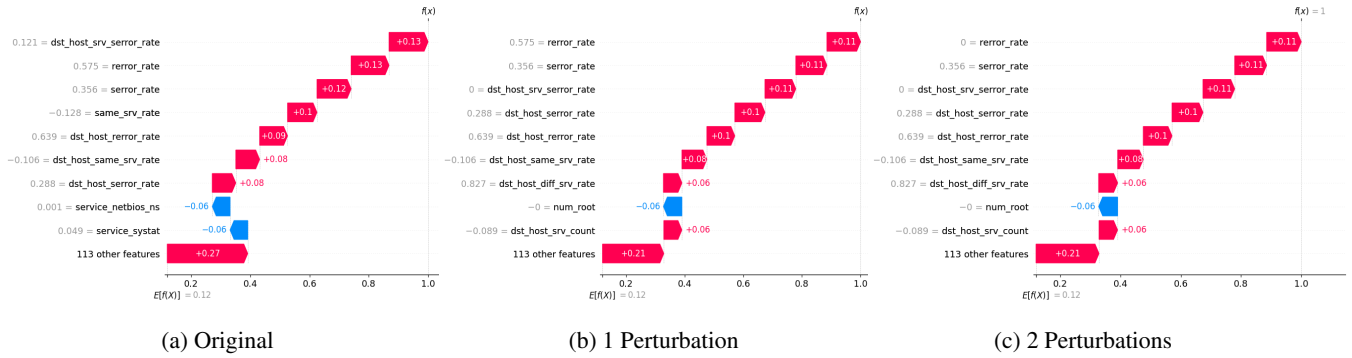


FIGURE 15: A local completeness example on a Probe sample from NSL-KDD dataset. This example is using SHAP.

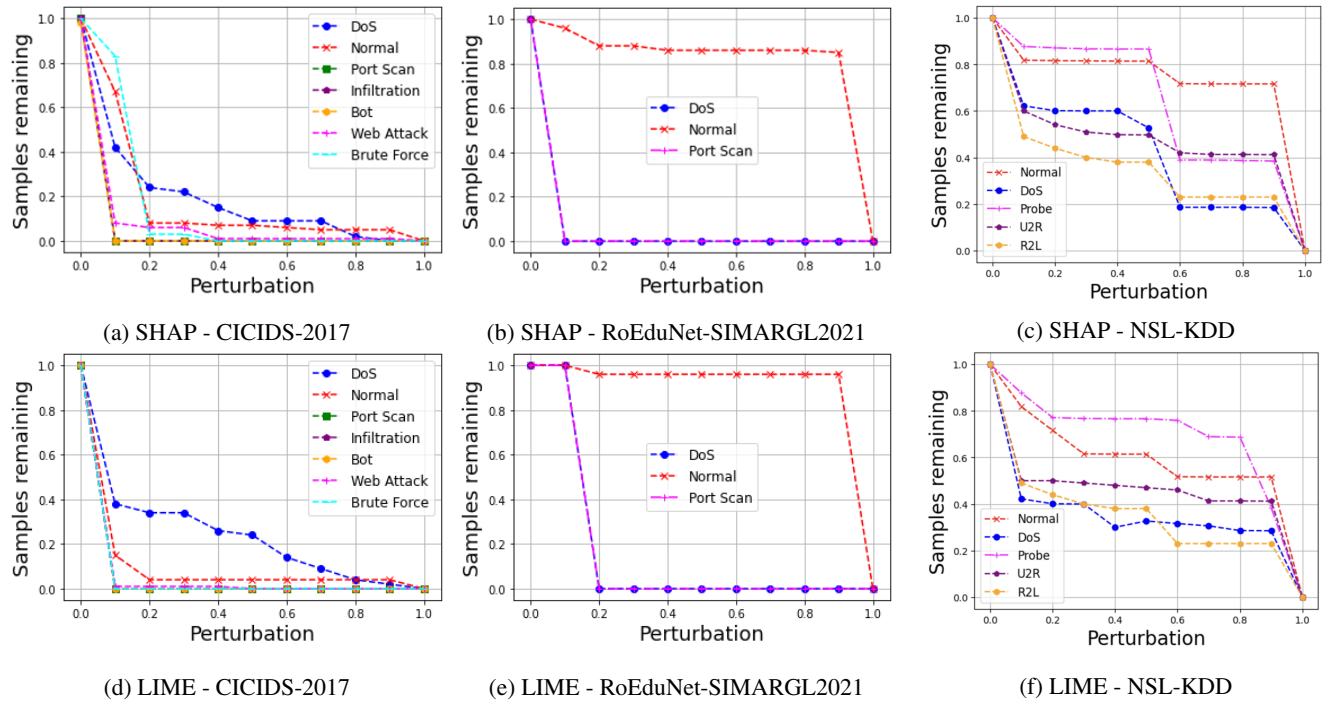


FIGURE 16: Percentage of remaining network traffic samples after removing the samples in which the intrusion class changed under different perturbation levels in intrusion features. We observe that higher perturbations tend to change classes for both LIME and SHAP for all three datasets.

security, and AI aspects) metrics thoroughly for three popular intrusion detection datasets and seven AI models. We also open-sourced our codes to contribute to the research community. Also, we successfully applied the metrics to all three datasets, with being considered a real-world dataset (RoEduNet-SIMARGL2021) and two testbed but well-renowned datasets (CICIDS-2017, and NSL-KDD) with two of the most widely used XAI methods (SHAP and LIME). This shows the potential of our work in enhancing the applicability of XAI methods for real-world IDS.

The findings show that LIME and SHAP cannot be used in their current forms in a production environment. The reason is that the metrics show some needed areas for improvement for both XAI techniques. An XAI method has to be efficient, in

other words, a security analyst cannot wait a long time to have access to an explanation while an attack might be occurring. Therefore, the results show that if one is using SHAP, it is not wise to pair it with KNN, SVM, MLP, and ADA. Meanwhile, LIME shows the same performance for the seven models. However, it is not optimized for global explanations.

This resolution leaves the RF, LGBM, and DNN models available to pair with LIME or SHAP. However, the Completeness and Robustness experiments show that LIME and SHAP do not have explanations for all the samples for the datasets. Although, they achieved Completeness for one or two classes. Neither SHAP nor LIME are complete considering all the samples analysed. The implication of this might be a case in which a security analyst has a scenario

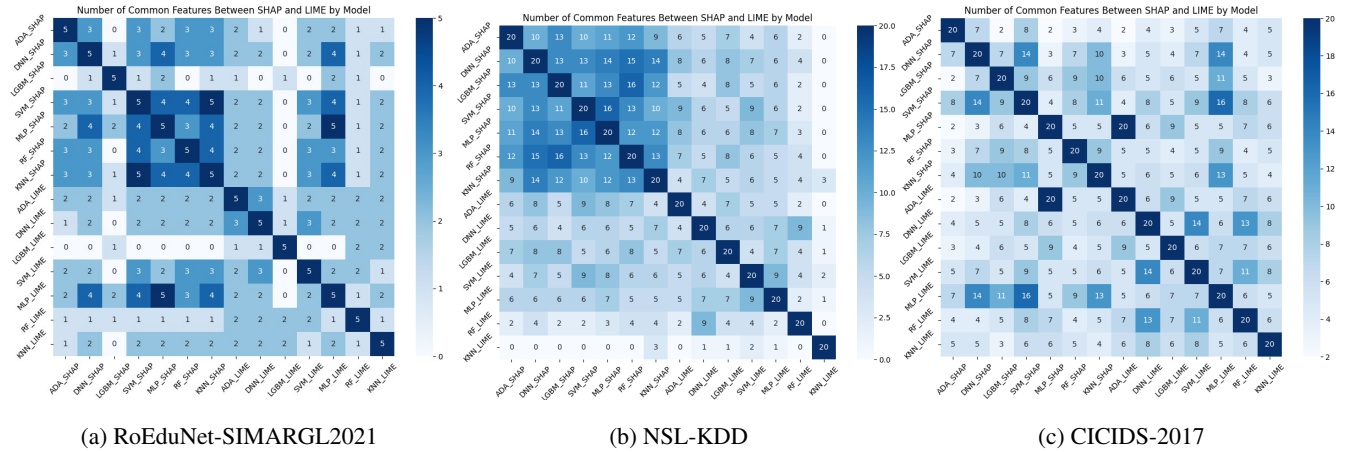


FIGURE 17: A heatmap analysis that shows the common top features across different AI models and the two XAI methods (SHAP and LIME) for all of the three studied datasets. This heatmap visualizes the overlap of top features identified by both SHAP and LIME across various AI models for each dataset. It also quantifies the number of matched features for each pair, highlighting the consistency in feature importance across different models and XAI techniques.

TABLE 17: This table shows the pairwise statistical test results between every pair of AI models by Wilcoxon signed rank test. Statistically better method ($p < 0.05$) shown in **bold** (both marked bold if there is significance and the median of the accuracies are the same. Only one is marked bold if there is significance and one model has a higher median). On the left, the CICIDS-2017 testbed is shown. In the middle, the RoEduNet-SIMARGL2021 testbed is shown. On the right, the NSL-KDD testbed is shown. For all testbeds, our method is statistically sound.

CICIDS-2017			RoEduNet-SIMARGL2021			NSL-KDD		
Model 1	Model 2	p-value	Model 1	Model 2	p-value	Method 1	Method 2	p-value
Random Forest	DNN	1.9073×10^{-6}	Random Forest	DNN	1.9073×10^{-6}	Random Forest	DNN	1.9073×10^{-6}
Random Forest	LGBM	0.9272	Random Forest	LGBM	0.0531	Random Forest	LGBM	0.0014
Random Forest	SVM	1.3351×10^{-4}	Random Forest	SVM	1.9073×10^{-6}	Random Forest	SVM	1.9073×10^{-6}
Random Forest	MLP	0.0695	Random Forest	MLP	0.0192	Random Forest	MLP	1.9073×10^{-6}
Random Forest	ADA	3.8146×10^{-6}	Random Forest	ADA	1.9073×10^{-6}	Random Forest	ADA	1.9073×10^{-6}
Random Forest	KNN	0.0003	Random Forest	KNN	1.9073×10^{-6}	Random Forest	KNN	1.9073×10^{-6}
DNN	LGBM	1.9073×10^{-6}	DNN	LGBM	1.9073×10^{-6}	DNN	LGBM	1.9073×10^{-6}
DNN	SVM	1.9073×10^{-6}	DNN	SVM	1.9073×10^{-6}	DNN	SVM	1.9073×10^{-6}
DNN	MLP	1.9073×10^{-6}	DNN	MLP	1.9073×10^{-6}	DNN	MLP	1.9073×10^{-6}
DNN	ADA	1.9073×10^{-6}	DNN	ADA	1.9073×10^{-6}	DNN	ADA	1.9073×10^{-6}
DNN	KNN	1.9073×10^{-6}	DNN	KNN	1.9073×10^{-6}	DNN	KNN	1.9073×10^{-6}
LGBM	SVM	0.0171	LGBM	SVM	0.0171	LGBM	SVM	0.8408
LGBM	MLP	0.0327	LGBM	MLP	0.0637	LGBM	MLP	1.9073×10^{-6}
LGBM	ADA	3.8146×10^{-6}	LGBM	ADA	1.9073×10^{-6}	LGBM	ADA	1.9073×10^{-6}
LGBM	KNN	0.0004	LGBM	KNN	0.6407	LGBM	KNN	1.9073×10^{-6}
SVM	MLP	9.536×10^{-6}	SVM	MLP	1.9073×10^{-6}	SVM	MLP	1.9073×10^{-6}
SVM	ADA	0.0896	SVM	ADA	1.9073×10^{-6}	SVM	ADA	1.9073×10^{-6}
SVM	KNN	1.9073×10^{-6}	SVM	KNN	1.9073×10^{-6}	SVM	KNN	1.9073×10^{-6}
MLP	ADA	1.9073×10^{-6}	MLP	ADA	1.9073×10^{-6}	MLP	ADA	1.9073×10^{-6}
MLP	KNN	0.0094	MLP	KNN	1.9073×10^{-6}	MLP	KNN	1.9073×10^{-6}
ADA	KNN	3.8146×10^{-6}	ADA	KNN	1.9073×10^{-6}	ADA	KNN	1.9073×10^{-6}

in which it is not explainable by the XAI method. Such a scenario might imply security breaches for the company, loss of trust among users and stakeholders, and it might fail audit procedures. From the Robustness side, the attacker can exploit the post-hoc XAI explanation method and intercept or switch the explanation. This is a risk that the security research community must be aware of and try to mitigate using other tools (or enhancing LIME and SHAP) since LIME and SHAP, in their current implementations, proved to be exploitable for these types of attacks.

I. EVALUATING PERFORMANCE OF CNN

We finally apply CNN to our performance metrics and the same XAI evaluation metrics as we did with the other

models. We stress that the work [23] used such models because their datasets consist of code fragments and infected documents (e.g., PDFs). In particular, this scenario is perfect for CNN and RNN since the data has an intrinsic positional relationship, and the order of information matters. Thus, it benefits from the ways that these models work.

Yet, the representation problem is usually solved because humans pre-define the characteristics of the tabular data. Since relationships between features in tabular data are not spatial, Convolutional Neural Network (CNN) may not offer a significant advantage over other models. Also, we stress that CNN, or filter-based techniques, work well for images due to their spatial inductive bias. They assume that pixels near each other relate to each other, which is

TABLE 18: A summary of performance comparison between SHAP and LIME for all six performance metrics. Overall, SHAP provides the best performance compared to LIME for the six metrics analyzed in this work.

Metric	Descriptive Accuracy	Sparsity	Stability	Completeness	Robustness	Efficiency
SHAP	✓	✓	✓	✓	✓	✓
LIME			✓		✓	✓

suitable for image data where spatial relationships matter. Finally, Tabular data is often heterogeneous, with little connection among features, different semantics, and varied units. Some techniques explored using CNNs for tabular data by converting it into images or 1-dimensional arrays. Hence, the application of CNNs in this context is not as straightforward as in image data, which is the approach we tried to do to address this comment concerning CNN.

Performance of CNN AI Model for Network Intrusion Detection: To successfully apply CNN, we lean into the work of [66]. This work transforms the network intrusion data into the array data format to apply it to the CNN model. Such a model consists of one Convolution 1D layer, pooling 1D layer, fully connected layer, and ReLU activation function. In sequence, the model is trained and evaluated for all datasets. See Table 19 below for performances of CNN for all of our three datasets. Such a table shows that CNN achieves high performance for CICIDS-2017 and RoEduNet-SIMARGL2021, while it has lower performance for the NSL-KDD dataset. Note that Table 19 shows all performance metrics.

Challenges for SHAP and LIME with CNN: We want to first stress some limitations we faced when applying XAI methods. The first challenge is data conversion. After explaining the samples using LIME and SHAP, the array values are converted back to the tabular format to be human-readable. This extra step of data conversion might be one more point of failure when considering real-world applications. Also, in another previous experiment based on [67], we faced the same issue when converting the samples for images. In this case, after training the CNN model and applying the SHAP explainer, we could not explain it anymore in a human-readable manner because the tabular dataset went through many transformations to RGB images. When applied to SHAP, it interprets each pixel as a single feature, making the explanation not understandable for humans.

Another challenge is resource utilization, especially for SHAP. When applying SHAP to samples, we faced memory utilization problems, often killing the Kernels of our Computer (256 GB RAM available). When generating explanations, we observed peaks of memory utilization that rendered this experiment hard to perform with the current conditions. This scenario allowed us to obtain the SHAP XAI metrics for CICIDS-2017 with features reduced by half and using only 100 samples for the XAI methods. Therefore, we consider this experiment only as a proof of concept that CNN works with SHAP. Nonetheless, we can use LIME without any resource issues, but for consistency, we limited

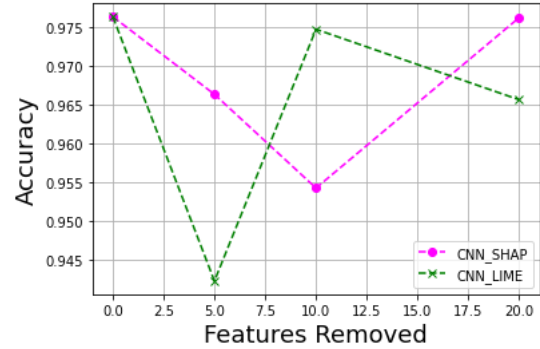


FIGURE 18: Descriptive Accuracy using the CNN model for the CICIDS-2017 dataset. Note that the plot indicates evidence that the most important features might not be influential because there is no significant drop in accuracy for both XAI techniques.

the experiments to 100 samples as well.

Evaluating XAI Metrics with CNN as AI Model: Next, we apply the XAI metrics to the datasets under CNN.

(i) **Descriptive Accuracy:** For this metric, SHAP, and LIME have similar performances as shown in Figure 18 in this section. In this case, the experiment shows that XAI feature selection did not impact the CNN's performance when removed. We respectively removed the most influential 5, 10, and 20 features, and the accuracy remained roughly the same throughout the graph. The interpretation of such a result is that the features selected were unimportant since the model can keep its accuracy high, and the CNN can use the information of the remaining features to make correct predictions.

(ii) **Sparsity:** For this metric, SHAP outperforms LIME as shown in Figure 19. The graph indicates that SHAP can select important features better than LIME. In other words, SHAP has more area under the curve, and its curve ascends sooner than LIME's.

(iii) **Efficiency:** LIME outperforms SHAP for CNN in time efficiency and memory efficiency. Often in our experiments, generating sample explanations for SHAP revolved around using high quantities of RAM (e.g., with CNN the experiments often went over 250 GB, rendering the kernel unusable). Such a problem did not happen with LIME. Regarding time efficiency, for 100 samples, LIME took 9 minutes, and SHAP took 276 minutes. Plus, experiments with fewer samples indicate that time complexity is linear for both XAI techniques under the CNN model.

(iv) **Stability:** Given the limited memory resources for SHAP circumstances, we did not perform the Stability measure for the technique. Nonetheless, considering the top

TABLE 19: CNN performance metrics (accuracy, precision, recall, F1, BACC, MCC, and AUC) for the three intrusion datasets.

Algorithm	Accuracy	Precision	Recall	F1	BACC	MCC	AUC_ROC
CICIDS-2017	0.9848	0.8277	0.6517	0.6641	0.6517	0.9761	0.9989
RoEduNet-SIMARGL2021	0.9976	0.9975	0.9971	0.9973	0.9971	0.9963	0.9971
NSL-KDD	0.7290	0.6516	0.493	0.4880	0.4929	0.5959	0.8946

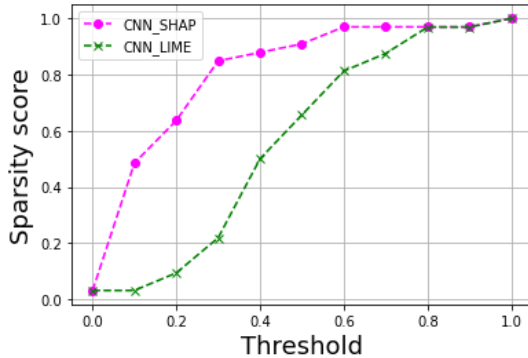


FIGURE 19: Sparsity metric using the CNN model for the CICIDS-2017 dataset. Note that the SHAP XAI method outperforms LIME because it has more area under the curve, and its curve ascends sooner.

10 features and repeating the experiment three times resulted in a Stability metric for LIME of 54.54%.

(v) **Completeness and Robustness:** We did not perform these experiments for CNN because we use Random Forest as a basis since we are looking for the validity of explanations for either SHAP or LIME. We kindly refer to our earlier analysis in this section for more details and a full analysis of these metrics.

In the interest of space, we only show the results for XAI metrics for CNN model for CICIDS-2017 dataset.

Discussion of Main Findings for CNN: Based on the results, we can conclude that LIME is better suited to work with the CNN setup used for this experiment. The reasons for this verdict are mainly the memory efficiency problem found when using SHAP paired with CNN, which makes the combination almost unusable and impractical. In the other spectrum, LIME does not display the same issue. Considering the XAI metrics, both LIME and SHAP perform the same in Descriptive Accuracy in which there is no significant drop in accuracy for both cases, which also might indicate the case that CNN might not be the best option for explainability when compared to the other seven models analyzed. In the case of Sparsity, SHAP performed better than LIME, indicating it selected better features. Analyzing the Time Efficiency, LIME is roughly 30 times faster than SHAP in the same conditions analyzed. Unfortunately, we could not derive the Stability experiment for SHAP due to its memory complexity issue. Nonetheless, LIME performed 54.54% for Stability.

VII. LIMITATIONS AND DISCUSSION

(1) **Importance of Our XAI Framework for IDS:** In

today's world of exponential information growth, it is natural that network attacks would become more frequent (e.g., see the recent study by the Center for Strategic & International Studies (CSIS) [68]). Although IDS has evolved throughout the years, security analysts are responsible for double-checking possible attack occurrences that are generated faster than they can be analyzed. Therefore, having accurate XAI-based intrusion detection systems can help solve such a problem. Our framework proposed in this work helps to evaluate these XAI-based systems thoroughly to deploy them for network security.

(2) **Exploring Our XAI Evaluation Framework on Other Benchmark Datasets:** In this work, we focused on giving a complete treatment of our XAI evaluation framework on three state-of-the-art network intrusion detection datasets (CICIDS-2017, NSL-KDD, and RoEduNet-SIMARGL2021). However, one fruitful direction for related future research is to explore the efficiency of our XAI evaluation framework on other network intrusion detection benchmark datasets, including UNSW-NB15 [69], UMass [70], and vulnerability-based datasets [71], and among others.

(3) **Dataset Analysis:** The experiments allow us to extract a few interesting conclusions about the datasets used. First, we conclude that the models achieved better explanations for the NSL-KDD dataset than the other two datasets. We can speculate that each NSL-KDD feature column information tends to be unique, and it does not overlap or have a higher correlation with other columns. Hence, models tend to find clear relations and patterns with a lower number of features for NSL-KDD. As a consequence, they can provide better XAI-based explanations. Second, we observe that NSL-KDD does not have fewer features that can bias the models used (as explained in the next subsection named "Effect of Bias of Top Intrusion Features"). Another possibility is that it could also mean that RF, LGBM, and KNN can produce high accuracy for only a handful number of features. However, this is a less likely option since this is only true for CICIDS-2017 and the RoEduNet-SIMARGL2021 datasets. Third, the size of datasets does not seem to correlate directly with performance, even when there is a significant difference in the number of features of the datasets (i.e., CICIDS-2017 has 78 features, RoEduNet-SIMARGL2021 has 29 features, and NSL-KDD has 41 features), the same models in CICIDS-2017 do not perform well for the NSL-KDD. Therefore, we could derive that the original number of features in the dataset is not the key factor for explanations.

(4) **Reliability of Current Black-box XAI Methods:** Although SHAP and LIME can be used for auditors when

gathering information and understanding network traffic logs for network IDS. Our work shows that the performance of SHAP and LIME would need to be improved to be used in real-world IDS. In particular, our work shows that it is desirable to enhance SHAP and LIME to be more robust against adversarial attacks. Furthermore, it shows the need to validate the completeness of explanations from SHAP and LIME before deploying them in reality.

(5) Development of Black-box XAI Methods: This work shows the possibility of having an enhanced black-box XAI tool for helping network security analysts, saving them time and increasing their efficiency and assertiveness. This paper shows that such a powerful XAI tool should eventually succeed in all the six metrics evaluated in this work. Throughout this paper, we checked positively that a few network intrusion classes can be explained completely by the XAI methods. At the same time, our robustness test showed weak points in post-hoc black-box XAI explanations. In particular, it suggests enhancing the robustness of XAI methods to make adversarial attacks difficult, notably when the OOD classifier can differentiate a perturbed generated sample from a network traffic sample. Moreover, the efficiency experiment shows the appropriate AI models to be paired with XAI methods (e.g., SHAP has better efficiency when paired with specific AI models). Similarly, descriptive accuracy, sparsity, and stability metrics also displayed proper consistency of XAI results under a proper selection of AI models. Overall, all these observations and suggestions can be used to pave the way toward the end goal of having a powerful black-box XAI tool for network IDS.

(6) Incorporation of LEMNA in Future XAI Evaluations: While our E-XAI framework currently focuses on SHAP and LIME, the exclusion of other XAI methods like LEMNA presents a limitation in our study. LEMNA, known for its effectiveness in unraveling complex nonlinear data relationships, could potentially offer valuable insights that are not captured by SHAP and LIME. However, its limited user accessibility and adaptability, particularly when compared to SHAP and LIME, along with its less extensive testing, raise concerns about its reliability and overall performance in network intrusion detection. Additionally, LEMNA's approach, which shares similarities with software fuzzing methods, introduces certain risks that need careful consideration. These risks akin to those found in software fuzzing methods that incorporate random noise into data inputs, which may pose risks such as generating false positives or negatives, potential overfitting issues, and susceptibility to adversarial attacks due to their exploratory nature. Such inherent randomness can deliver unreliable results. Future research should aim to include LEMNA and similar methods, addressing these challenges to provide a more rounded evaluation of XAI tools for network security.

(7) Addressing Feature Bias in AI Models: Concerning overall features, prior works show that some features demonstrate bias in the dataset generation. In particular, "Destination Port" has been revealed to be a sign of bias [72]

where it can have a contaminating effect when included in AI network intrusion detection models. Similarly, we may infer that TCP_WIN_SCALE_IN may have the same contaminating effect for the RoEduNet-SIMARGL2021 dataset. We emphasize that our decision-making XAI explanation and feature importance do not depend on a single feature. Thus, even if some features are biased, the majority should compensate for such a bias. Another possibility for accommodating this is adding a warning with the XAI explanation if one top feature is from these two biased features. Incorporating warnings or indicators in XAI explanations about potentially biased top features could help users better understand and interpret the results, leading to more informed decision-making.

(8) Time Performance Variations Among XAI Methods and AI Models: Our analysis reveals significant differences in the time efficiency of various AI models when coupled with XAI methods like SHAP and LIME. LIME shows general time efficiency across different models but falls short with KNN, while SHAP pairs better with models such as LGBM, RF, and DNN, but is less efficient with MLP, ADA, and KNN. This highlights the crucial need for selecting the appropriate combination of AI models and XAI methods based on specific requirements and real-time operational demands. Future iterations of our proposed E-XAI framework should aim to optimize this aspect, ensuring that the chosen XAI methods not only provide accurate and understandable explanations but also do so within the time constraints typical of real-world network intrusion detection scenarios.

VIII. CONCLUSION

The end goal of a security intrusion detection tool is to serve as a shield or as a prevention mechanism against intruders. The usage of AI can help in having these tools automated. Thus, there is a pressing need to find new solutions that can effectively explain AI intrusion detection methods. This paper presented an end-to-end framework for the evaluation of black-box XAI techniques for network intrusion detection tasks. Our framework evaluated both global and local explanations for XAI methods for network IDS. We analyzed six different evaluation metrics for two popular black-box XAI techniques (SHAP and LIME).

These six metrics are descriptive accuracy (for checking trade-offs between XAI explainability and intrusion detection accuracy), sparsity (for measuring the correlation between different intrusion features given by the XAI method), stability (for checking the consistency of the XAI method in generating its explanations for traffic instances), efficiency (for measuring the applicability of the XAI method in real-world systems), robustness (for checking the ability of XAI method in generating the same explanations under adversarial attacks), and completeness (for testing the ability of XAI method in providing a correct explanation for every possible network traffic sample, including corner cases).

We evaluated our XAI evaluation framework via extensive experiments using three popular network intrusion datasets

(RoEduNet-SIMARGL2021, CICIDS-2017, and NSL-KDD) and under seven different AI models. We released our source codes for the research community to access it as a baseline XAI evaluation framework for network intrusion detection and to build on it with new datasets and models. This work represents a critical step towards applying black-box XAI methods for real-world network IDS via understanding the strengths and limitations of current black-box XAI methods when applied to network IDS.

ACKNOWLEDGMENT

This work is supported in part by AnalytixIN, EMPOWER, and 1st Year Research Immersion Program (IRIP) grants from the office of the Vice Chancellor for Research at Indiana University-Purdue University Indianapolis.

REFERENCES

- [1] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications surveys & tutorials*, vol. 18, no. 2, pp. 1153–1176, 2015.
- [2] A. S. Dina and D. Manivannan, "Intrusion detection based on machine learning techniques in computer networks," *Internet of Things*, vol. 16, p. 100462, 2021.
- [3] J. Kim, N. Shin, S. Y. Jo, and S. H. Kim, "Method of intrusion detection using deep neural network," in *2017 IEEE international conference on big data and smart computing (BigComp)*. IEEE, 2017, pp. 313–316.
- [4] C. Tang, N. Luktaran, and Y. Zhao, "Saae-dnn: Deep learning method on intrusion detection," *Symmetry*, vol. 12, no. 10, p. 1695, 2020.
- [5] P. Tao, Z. Sun, and Z. Sun, "An improved intrusion detection algorithm based on ga and svm," *Ieee Access*, vol. 6, pp. 13 624–13 631, 2018.
- [6] H. Deng, Q.-A. Zeng, and D. P. Agrawal, "Svm-based intrusion detection system for wireless ad hoc networks," in *2003 IEEE 58th Vehicular Technology Conference. VTC 2003-Fall (IEEE Cat. No. 03CH37484)*, vol. 3. IEEE, 2003, pp. 2147–2151.
- [7] M. A. Ferrag, L. Maglaras, A. Ahmim, M. Dourdour, and H. Janicke, "Rtdits: Rules and decision tree-based intrusion detection system for internet-of-things networks," *Future internet*, vol. 12, no. 3, p. 44, 2020.
- [8] M. Al-Omari, M. Rawashdeh, F. Qutaishat, M. Alshira'H, and N. Ababneh, "An intelligent tree-based intrusion detection model for cyber security," *Journal of Network and Systems Management*, vol. 29, no. 2, pp. 1–18, 2021.
- [9] N. B. Amor, S. Benferhat, and Z. Elouedi, "Naive bayes vs decision trees in intrusion detection systems," in *Proceedings of the 2004 ACM symposium on Applied computing*, 2004, pp. 420–424.
- [10] R. Panigrahi, S. Borah, M. Pramanik, A. K. Bhoi, P. Barsocchi, S. R. Nayak, and W. Alnumay, "Intrusion detection in cyber-physical environment using hybrid naive bayes—decision table and multi-objective evolutionary feature selection," *Computer Communications*, vol. 188, pp. 133–144, 2022.
- [11] A. K. Balyan, S. Ahuja, U. K. Lilhore, S. K. Sharma, P. Manoharan, A. D. Algarni, H. Elmannai, and K. Raahemifar, "A hybrid intrusion detection model using ega-psi and improved random forest method," *Sensors*, vol. 22, no. 16, p. 5986, 2022.
- [12] S. Waskle, L. Parashar, and U. Singh, "Intrusion detection system using pca with random forest approach," in *2020 International Conference on Electronics and Sustainable Communication Systems (ICESC)*. IEEE, 2020, pp. 803–808.
- [13] S. Arisdakessian, O. A. Wahab, A. Mourad, H. Otrouk, and M. Guizani, "A survey on iot intrusion detection: Federated learning, game theory, social psychology and explainable ai as future directions," *IEEE Internet of Things Journal*, 2022.
- [14] S. I. Savev, "Integrated approach to cyber defence: Human in the loop. technical evaluation report," *Information & Security: An International Journal*, vol. 44, pp. 76–92, 2020.
- [15] A. Das and P. Rad, "Opportunities and challenges in explainable artificial intelligence (xai): A survey," *arXiv preprint arXiv:2006.11371*, 2020.
- [16] B. Mahbooba, M. Timilsina, R. Sahal, and M. Serrano, "Explainable artificial intelligence (xai) to enhance trust management in intrusion detection systems using decision tree model," *Complexity*, vol. 2021, 2021.
- [17] S. Patil, V. Varadarajan, S. M. Mazhar, A. Sahibzada, N. Ahmed, O. Sinha, S. Kumar, K. Shaw, and K. Kotecha, "Explainable artificial intelligence for intrusion detection system," *Electronics*, vol. 11, no. 19, p. 3079, 2022.
- [18] S. R. Islam, W. Eberle, S. K. Ghafoor, A. Siraj, and M. Rogers, "Domain knowledge aided explainable artificial intelligence for intrusion detection and response," *arXiv preprint arXiv:1911.09853*, 2019.
- [19] E. Roponen, J. Kampars, J. Grabis, and A. Gailitis, "Towards a human-in-the-loop intelligent intrusion detection system," in *CEUR Workshop Proceedings*, 2022, pp. 71–81.
- [20] D. Han, Z. Wang, W. Chen, K. Wang, R. Yu, S. Wang, H. Zhang, Z. Wang, M. Jin, J. Yang *et al.*, "Anomaly detection in the open world: Normality shift detection, explanation, and adaptation," in *30th Annual Network and Distributed System Security Symposium (NDSS)*, 2023.
- [21] L. Dhanabal and S. Shantharajah, "A study on nsl-kdd dataset for intrusion detection system based on classification algorithms," *International journal of advanced research in computer and communication engineering*, vol. 4, no. 6, pp. 446–452, 2015.
- [22] J. Dieber and S. Kirrane, "Why model why? assessing the strengths and limitations of lime," *arXiv preprint arXiv:2012.00093*, 2020.
- [23] A. Warnecke, D. Arp, C. Wressnegger, and K. Rieck, "Evaluating explanation methods for deep learning in security," in *2020 IEEE european symposium on security and privacy (EuroS&P)*. IEEE, 2020, pp. 158–174.
- [24] D. Slack, S. Hilgard, E. Jia, S. Singh, and H. Lakkaraju, "Fooling lime and shap: Adversarial attacks on post hoc explanation methods," in *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*, 2020, pp. 180–186.
- [25] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," *CoRR*, vol. abs/1602.04938, 2016. [Online]. Available: <http://arxiv.org/abs/1602.04938>
- [26] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable ai for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 2522–2539, 2020.
- [27] M. Wang, K. Zheng, Y. Yang, and X. Wang, "An explainable machine learning framework for intrusion detection systems," *IEEE Access*, vol. 8, pp. 73 127–73 141, 2020.
- [28] C. Wu, A. Qian, X. Dong, and Y. Zhang, "Feature-oriented design of visual analytics system for interpretable deep learning based intrusion detection," in *2020 International Symposium on Theoretical Aspects of Software Engineering (TASE)*, 2020, pp. 73–80.
- [29] M.-E. Mihailescu, D. Mihai, M. Carabas, M. Komisarek, M. Pawlicki, W. Holubowicz, and R. Kozik, "The proposition and evaluation of the roedunet-simargl2021 network intrusion detection dataset," *Sensors*, vol. 21, no. 13, p. 4319, 2021.
- [30] R. Panigrahi and S. Borah, "A detailed analysis of cicids2017 dataset for designing intrusion detection systems," *International Journal of Engineering & Technology*, vol. 7, no. 3.24, pp. 479–482, 2018.
- [31] D. Han, Z. Wang, W. Chen, Y. Zhong, S. Wang, H. Zhang, J. Yang, X. Shi, and X. Yin, "Deepaid: Interpreting and improving deep learning-based anomaly detection in security applications," 09 2021.
- [32] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," 2018.
- [33] S. Neupane, J. Ables, W. Anderson, S. Mittal, S. Rahimi, I. Banicescu, and M. Seale, "Explainable intrusion detection systems (x-ids): A survey of current methods, challenges, and opportunities," 2022.
- [34] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "Mitre att&ck: Design and philosophy," in *Technical report*. The MITRE Corporation, 2018.
- [35] C. B. Lee, C. Roedel, and E. Silenok, "Detection and characterization of port scan attacks," *University of California, Department of Computer Science and Engineering*, 2003.
- [36] Kurniabudi, D. Stiawan, Darmawijoyo, M. Y. Bin Idris, A. M. Bamhdi, and R. Budiarto, "Cicids-2017 dataset feature analysis with information gain for anomaly detection," *IEEE Access*, vol. 8, pp. 132 911–132 921, 2020.
- [37] M.-E. Mihailescu, D. Mihai, M. Carabas, M. Komisarek, M. Pawlicki, W. Holubowicz, and R. Kozik, "The proposition and evaluation of the roedunet-simargl2021 network intrusion detection dataset," *Sensors*, vol. 21, no. 13, 2021.
- [38] D. by Comprmoise, "Drive-by Compromise," <https://attack.mitre.org/techniques/T1189/>, 2023, [Online; accessed 21-October-2023].
- [39] B. Stone-Gross, M. Cova, L. Cavallaro, B. Gilbert, M. Szydlowski, R. Kemmerer, C. Kruegel, and G. Vigna, "Your botnet is my botnet: analysis of a botnet takeover," in *Proceedings of the 16th ACM conference on Computer and communications security*, 2009, pp. 635–647.

- [40] Y. Chen, Q. Lin, W. Wei, J. Ji, K.-C. Wong, and C. A. Coello Coello, "Intrusion detection using multi-objective evolutionary convolutional neural network for internet of things in fog computing," *Knowledge-Based Systems*, vol. 244, p. 108505, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950705122002179>
- [41] V. Gorodetski and I. Kotenko, "Attacks against computer network: Formal grammar-based framework and simulation tool," in *International Workshop on Recent Advances in Intrusion Detection*. Springer, 2002, pp. 219–238.
- [42] M. Skwarek, M. Korczynski, W. Mazurczyk, and A. Duda, "Characterizing vulnerability of dns axfr transfers with global-scale scanning," in *2019 IEEE Security and Privacy Workshops (SPW)*. IEEE, 2019, pp. 193–198.
- [43] A. Khan, H. Kim, and B. Lee, "M2mon: Building an mmio-based security reference monitor for unmanned vehicles." 2021.
- [44] S. R. Hussain, I. Karim, A. A. Ishtiaq, O. Chowdhury, and E. Bertino, "Noncompliance as deviant behavior: An automated black-box noncompliance checker for 4g lte cellular devices," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 1082–1099.
- [45] O. Mirzaei, R. Vasilenko, E. Kirda, L. Lu, and A. Kharraz, "Scrutinizer: Detecting code reuse in malware via decompilation and machine learning," in *Detection of Intrusions and Malware, and Vulnerability Assessment: 18th International Conference, DIMVA 2021, Virtual Event, July 14–16, 2021, Proceedings 18*. Springer, 2021, pp. 130–150.
- [46] S. Lukacs, D. H. Lutas, A. V. COLESA *et al.*, "Strongly isolated malware scanning using secure virtual containers," Aug. 25 2015, uS Patent 9,117,081.
- [47] A. Kim, M. Park, and D. H. Lee, "Ai-ids: Application of deep learning to real-time web intrusion detection," *IEEE Access*, vol. 8, pp. 70 245–70 261, 2020.
- [48] M. Botacin, F. Ceschin, R. Sun, D. Oliveira, and A. Grégio, "Challenges and pitfalls in malware research," *Computers & Security*, vol. 106, p. 102287, 2021.
- [49] I. Amit, J. Matherly, W. Hewlett, Z. Xu, Y. Meshi, and Y. Weinberger, "Machine learning in cyber-security - problems, challenges and data sets," 2018. [Online]. Available: <https://arxiv.org/abs/1812.07858>
- [50] N. Capuano, G. Fenza, V. Loia, and C. Stanzione, "Explainable artificial intelligence in cybersecurity: A survey," *IEEE Access*, vol. 10, pp. 93 575–93 600, 2022.
- [51] B. M. Greenwell, "pdp: an r package for constructing partial dependence plots." *R J.*, vol. 9, no. 1, p. 421, 2017.
- [52] A. Goldstein, A. Kapelner, J. Bleich, and E. Pitkin, "Peeking inside the black box: Visualizing statistical learning with plots of individual conditional expectation," *Journal of Computational and Graphical Statistics*, vol. 24, no. 1, pp. 44–65, 2015.
- [53] E. Anderssen, K. Dyrstad, F. Westad, and H. Martens, "Reducing over-optimism in variable selection by cross-model validation," *Chemometrics and intelligent laboratory systems*, vol. 84, no. 1-2, pp. 69–74, 2006.
- [54] "Flow information elements - nprobe 10.1 documentation." [Online]. Available: https://www.ntop.org/guides/nprobe/flow_information_elements.html
- [55] Ahlaskari, "Cicflowmeter/readme.txt at master · ahlaskari/cicflowmeter, <https://github.com/ahlaskari/cicflowmeter/blob/master/readme.txt>," Jun 2021. [Online]. Available: <https://github.com/ahlaskari/CICFlowMeter/blob/master/ReadMe.txt>
- [56] B. Claise, "Cisco systems netflow services export version 9," Tech. Rep., 2004.
- [57] I. Sharafaldin, A. Gharib, A. H. Lashkari, and A. A. Ghorbani, "Towards a reliable intrusion detection benchmark dataset," *Software Networking*, vol. 2018, no. 1, pp. 177–200, 2018.
- [58] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the kdd cup 99 data set," in *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 2009, pp. 1–6.
- [59] R. ZHAO, "Nsl-kdd," 2022. [Online]. Available: <https://dx.doi.org/10.21227/8rpg-qt98>
- [60] C. A. Stewart, V. Welch, B. Plale, G. C. Fox, M. Pierce, and T. Sterling, "Indiana university pervasive technology institute," techreport, 2017.
- [61] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in neural information processing systems*, vol. 30, 2017.
- [62] A. Yulianto, P. Sukarno, and N. A. Suwastika, "Improving adaboost-based intrusion detection system (ids) performance on cic ids 2017 dataset," in *Journal of Physics: Conference Series*, vol. 1192, no. 1. IOP Publishing, 2019, p. 012018.
- [63] W. Li, P. Yi, Y. Wu, L. Pan, and J. Li, "A new intrusion detection system based on knn classification algorithm in wireless sensor network," *Journal of Electrical and Computer Engineering*, vol. 2014, 2014.
- [64] J. O. Mebawodu, O. D. Alowolodu, J. O. Mebawodu, and A. O. Adetunmbi, "Network intrusion detection system using supervised learning paradigm," *Scientific African*, vol. 9, p. e00497, 2020.
- [65] D. Jin, Y. Lu, J. Qin, Z. Cheng, and Z. Mao, "Swiftids: Real-time intrusion detection system based on lightgbm and parallel intrusion detection mechanism," *Computers & Security*, vol. 97, p. 101984, 2020.
- [66] X. Zhang, J. Ran, and J. Mi, "An intrusion detection system based on convolutional neural network for imbalanced network traffic," in *2019 IEEE 7th International Conference on Computer Science and Network Technology (ICCSNT)*, 2019, pp. 456–460.
- [67] L. Yang and A. Shami, "A transfer learning and optimized cnn based intrusion detection system for internet of vehicles," in *ICC 2022 - IEEE International Conference on Communications*, 2022, pp. 2774–2779.
- [68] I. Insights, "42 Cyber Attack Statistics by Year: A Look at the Last Decade," <https://sectigostore.com/blog/42-cyber-attack-statistics-by-year-a-look-at-the-last-decade/>, February 2020, [Online; accessed 10-March-2023].
- [69] N. Moustafa and J. Slay, "Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set)," in *2015 military communications and information systems conference (MilCIS)*. IEEE, 2015, pp. 1–6.
- [70] U. T. Repository, "UMass Trace Repository," <http://traces.cs.umass.edu/index.php/Network/Network>, 2021, [Online; accessed on 21-November-2022].
- [71] Y. Dong, W. Guo, Y. Chen, and X. Xing, "Towards the detection of inconsistencies in public security vulnerability reports."
- [72] L. D'hooge, M. Verkerken, B. Volckaert, T. Wauters, and F. De Turck, "Establishing the contaminating effect of metadata feature inclusion in machine-learned network intrusion detection models," in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2022, pp. 23–41.
- [73] D. Chicco and G. Jurman, "The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation," *BMC genomics*, vol. 21, pp. 1–13, 2020.

APPENDIX A AI MODELS AND HYPER-PARAMETERS

We now provide the hyperparameters of our different AI models considered in this work.

A. AI MODELS AND HYPERPARAMETERS DETAILS

Deep Neural Network (DNN): The first classifier is a deep neural network (DNN). The architecture of this classifier consists of an input layer with the count of neurons corresponding to the used number of features, and the rectified linear unit (ReLU) activation function. This is followed by a dropout layer with the dropout set at 0.01, a hidden layer with a size of 16 neurons, and the ReLU activation function. The setup closes with a "softmax". The loss function was set to the "categorical_crossentropy" method, while the chosen optimization algorithm was adaptive momentum (ADAM). Eleven epochs were needed to train the model with a batch size of 1024.

Random Forest (RF): The next classifier used to detect malicious samples in the network traffic was the RandomForest (RF). The hyperparameters we used for this classifier are as follows: `n_estimators` (a parameter that signifies number of trees used) was set to the value of 100, the maximum tree depth was set to the value of 10, the minimum number of samples required to separate internal node was set to the value of 2, and the rest were used as provided by default.

AdaBoost (ADA): The next classifier used in this study was AdaBoost. The parameter configuration for this classifier is as follows: the maximum number of estimators at which boosting will be completed was set to a value of 50, the weight applied to each classifier in each boosting iteration was set to a value of 1, and the base estimator from which the boosted ensemble is built was set to Decision_Tree_Classifier.

K-nearest Neighbour (KNN): We also used the KNN classifier. For this study, we used KNN in its default hyperparameters as follows: the 'n' neighbors value was set to five, all weights were uniform, and the search algorithm was set to 'auto'.

Support Vector Machine (SVM): We also used the SVM classifier. The parameter configuration for this classifier is as follows: Kernel is set to 'linear', gamma to 0.5, the probability is set to 'True', and regularization is set to 0.5.

Multi-layer Perceptron (MLP): The next used classifier was MLP. We used the same setup as DNN for this classifier.

LightGBM: The next used classifier was LightGBM. We used it in its default hyper-parameters as follows: the n_splits was set to 10, n_repeats was set to 3, error_score was set to 'raise', n_jobs set to 1, and scoring was set to 'accuracy'.

B. HYPER-PARAMETER RATIONALE

For this work, we performed hyper-parameter tuning to maximize the performance of AI-based (or ML-based) classifiers. The rationale behind the hyper-parameter tuning in Appendix A is trial and error. Before applying the post-model XAI techniques in this work, we extensively trained and tested the same model with different parameters, checking its performance on a standard set of well-known metrics to evaluate each AI model for intrusion detection classification problem. These are accuracy (Acc), precision (Prec), recall (Rec), F1-score (F1), Matthews correlation coefficient (Mcc) [73], and balanced accuracy (Bacc), and the AucRoc (area under the ROC curve). For instance, taking the Random Forest model, we first checked what tree depth starts to get the best performance, then we experimented with the number of nodes for that tree, the splits for each node, and the number of trees, for example. The same is true for the other models as well. During this experimentation phase, we keep the better parameters found.

APPENDIX B LIST OF TOP INTRUSION FEATURES

We show the list of top intrusion features for both LIME and SHAP XAI methods for different datasets in Tables 27-28. In particular, Table 27 shows such top features for RoEduNet-SIMARGL2021 and CICIDS-2017 datasets under SHAP and LIME XAI methods. On the other hand, Table 28 shows such top features for NSL-KDD datasets.

APPENDIX C AI MODELS' PERFORMANCES UNDER ALL INTRUSION FEATURES

We used all intrusion features to calculate Accuracy, Precision, Recall, F1, Balanced Accuracy, Matthew Correlation Coefficient, and AUCROC for all AI models in

our work. We show such performances below in Tables 20-22. The results show that LightGBM, KNN, and RF have the best performances for CICIDS-2017 dataset. It also show that KNN, and MLP give the best performances for NSL-KDD dataset. Finally, it show that LightGBM, RF, and KNN have the best performances for RoEduNet-SIMARGL2021 dataset. Overall, using all features indicate that KNN and RF can be the best option if no feature importance analysis is performed for these three datasets.

APPENDIX D AI MODELS' PERFORMANCES UNDER SAME NUMBER OF FEATURES

In addition to testing models and XAI methods under all features, we have also tested the performances for all models under the same features where we choose the overall top eight features. These eight top features employed in our analysis are detailed in Table 23 below. Our results for such an experiment are shown in Tables 24-26 below. As shown in Table 24, the results for the NSL-KDD dataset reveal that the Support Vector Machine (SVM) algorithm notably excels, especially in terms of AUCROC, thereby underscoring its exceptional capability for in-class differentiation. This is in contrast to the comparatively moderate performance observed in Adaptive Boosting (ADA) and Multilayer Perceptron (MLP) across all datasets. In a different vein, K-Nearest Neighbors (KNN) and Support Vector Machine (SVM) demonstrate remarkable effectiveness in the CICIDS-2017 and RoEduNet-SIMARGL datasets, as shown in Tables 25 and 26 respectively, achieving near-perfect scores across several metrics. Nevertheless, their subdued performance in the NSL-KDD dataset suggests a potential sensitivity to specific data characteristics.

The varying performance of these algorithms across different datasets highlights the influence of dataset complexity and inherent characteristics on algorithm effectiveness. It becomes evident that while some algorithms like KNN and RF can excel in certain scenarios, their applicability might be limited in others, underscoring the importance of choosing the right algorithm based on the dataset at hand. This analysis, grounded in a consistent methodological approach, underscores the need for careful algorithm selection and validation in diverse data environments to ensure robust and generalizable machine learning models for network intrusion detection tasks.

APPENDIX E SHAP GLOBAL SUMMARY PLOTS

We here show the global summary plots of SHAP in Figures 20-22 for the RoEduNet-SIMARGL2021, CICIDS-2017, and NSL-KDD datasets, respectively. They show the most influential features in the AI model's decision in descending order. They display the outcome of SHAP global explanation by presenting horizontal bar charts. In these visuals, attacks are depicted as stacked bars, where the length signifies their SHAP score or contribution value to the model's predictions. Additionally, each row in the graph features bars assigned to individual features, arranged in descending order of importance. The diverse colors within

TABLE 20: Algorithm Metrics Comparison for all features for SHAPILIME in the NSL-KDD dataset. We observe that KNN and MLP yield the best performances across all AI models for both XAI methods.

Algorithm	Accuracy	Precision	Recall	F1	BACC	MCC	AUC_ROC
ADA	0.8259	0.5649	0.5649	0.5649	0.7280	0.4561	0.6251
DNN	0.8857	0.5160	0.4829	0.4531	0.6275	0.2533	0.6275
KNN	0.9962	0.9907	0.9907	0.9907	0.9942	0.9884	0.9361
Lightgbm	0.8410	0.6027	0.6027	0.6027	0.7517	0.5034	0.9446
MLP	0.9039	0.7598	0.7598	0.7598	0.8499	0.6998	0.7312
RF	0.8203	0.5509	0.5509	0.5509	0.7193	0.4386	0.5977
SVM	0.8619	0.6548	0.6548	0.6548	0.7842	0.5685	0.6332

TABLE 21: Algorithm Metrics Comparison for all features for SHAPILIME in the CICIDS-2017 dataset. We observe that KNN, LightGBM, and RF yield the best performances across all AI models for both XAI methods.

Algorithm	Accuracy	Precision	Recall	F1	BACC	MCC	AUC_ROC
ADA	0.7508	0.1279	0.1279	0.1279	0.4913	0.0173	0.9642
DNN	0.7624	0.1686	0.1686	0.1686	0.5150	0.0300	0.9750
KNN	0.9990	0.9924	0.9924	0.9924	0.9954	0.9909	0.9977
Lightgbm	0.9999	0.9997	0.9997	0.9997	0.9998	0.9997	0.9999
MLP	0.7143	0.0003	0.0003	0.0003	0.4168	0.1662	0.9965
RF	0.9791	0.9268	0.9268	0.9268	0.9573	0.9146	0.5995
SVM	0.9618	0.8665	0.8665	0.8665	0.9221	0.8442	0.8675

TABLE 22: Algorithm Metrics Comparison for all features for SHAP and LIME for the RoEduNet-SIMARGL2021 dataset. We observe that KNN, LightGBM, and RF yield the best performances across all AI models for both XAI methods.

Algorithm	Accuracy	Precision	Recall	F1	BACC	MCC	AUC_ROC
ADA	0.6704	0.5056	0.5056	0.5056	0.6292	0.25843	0.3657
DNN	0.7776	0.6664	0.6664	0.6664	0.7498	0.4996	0.5051
KNN	0.9988	0.9981	0.9981	0.9981	0.9985	0.9971	0.9997
Lightgbm	0.9997	0.9995	0.9995	0.9995	0.9996	0.9993	0.9994
MLP	0.3966	0.0949	0.0949	0.0949	0.3212	0.3575	0.9933
RF	0.9977	0.9960	0.9960	0.9960	0.9970	0.9940	0.9997
SVM	0.9153	0.8979	0.8979	0.8979	0.9234	0.8468	0.9713

TABLE 23: An overall feature importance (rank in descending order) across all AI models for RoEduNet-SIMARGL2021, CICIDS-2017, and NSL-KDD datasets.

Feature Rank	CICIDS-2017	RoEduNet-SIMARGL2021	NSL-KDD
1	Destination Port	TCP_WIN_MSS_IN	dst_host_same_srv_rate
2	Init_Win_Bytes_Backward	TCP_WIN_SCALE_IN	dst_host_srv_count
3	Packet Length Std	TCP_WIN_MAX_IN	dst_host_same_src_port_rate
4	Bwd Packet Length Mean	TCP_WIN_MIN_IN	logged_in
5	Total length of Bwd Packets	TCP_WIN_WIN_OUT	dst_host_serror_rate
6	Packet Length Mean	TCP_FLAGS	count
7	Subflow Backward Bytes	PROTOCOL	srv_count
8	Packet Length Variance	FLOW_DURATION_MILLISECONDS	dst_host_rerror_rate

each bar correspond to different attack classes, illustrating the impact of each class on the overall significance of the feature. For instance, a longer section of a specific color in a feature's bar indicates greater relevance of the associated attack class in the model's decision-making for that feature. This sophisticated visualization not only allows us to discern the overall global importance of each feature in the AI model but also highlights the distinct contribution of each feature for each attack class separately. In summary, one can extract from these graphs the model-specific top features and

attack-specific features as well.

TABLE 24: Algorithm Metrics Comparison for NSL-KDD under top same features.

Algorithm	Accuracy	Precision	Recall	F1	BACC	MCC	AUC_ROC
ADA	0.8411	0.3974	0.0526	0.7663	0.7757	0.5645	0.6524
DNN	0.8258	0.4356	0.1829	0.7653	0.7730	0.5644	0.6276
KNN	0.8229	0.1107	0.0111	0.7544	0.7590	0.5424	0.6167
Lightgbm	0.8351	0.1539	0.1418	0.7376	0.7326	0.5147	0.5942
MLP	0.8243	0.1754	0.0312	0.7493	0.7499	0.5361	0.6122
RF	0.8393	0.1804	0.0429	0.7451	0.7434	0.5293	0.6111
SVM	0.8545	0.2655	0.3248	0.7451	0.7429	0.5303	0.6006

TABLE 25: Algorithm Comparison for CICIDS-2017 under top same features.

Algorithm	Accuracy	Precision	Recall	F1	BACC	MCC	AUC_ROC
ADA	0.9507	0.8026	0.8026	0.8026	0.8872	0.7744	0.7057
DNN	0.9517	0.8068	0.8068	0.8068	0.8896	0.7792	0.6652
KNN	0.9981	0.9981	0.9981	0.9981	0.9981	0.9963	0.9993
Lightgbm	0.8880	0.6080	0.6080	0.6080	0.7713	0.5427	0.6870
MLP	0.9440	0.7762	0.7762	0.7762	0.8721	0.7442	0.7698
RF	0.9529	0.8351	0.8351	0.9038	0.8077	0.6054	0.3980
SVM	0.9812	0.9154	0.9154	0.9154	0.9524	0.9048	0.8123

TABLE 26: Algorithm Metrics Comparison for RoEduNet-SIMARGL under top same features.

Algorithm	Accuracy	Precision	Recall	F1	BACC	MCC	AUC_ROC
ADA	0.4870	0.2304	0.2304	0.2304	0.4228	-0.1543	0.5000
DNN	0.5666	0.3499	0.3499	0.3499	0.5124	0.0249	0.5132
KNN	0.9998	0.9998	0.9998	0.9998	0.9998	0.9996	0.9999
Lightgbm	0.9997	0.9996	0.9996	0.9996	0.9997	0.9994	1.0000
MLP	0.556	0.3333	0.3333	0.3333	0.5	0.0	0.9998
RF	0.9992	0.9988	0.9988	0.9988	0.9991	0.9982	1.0000
SVM	0.9932	0.9898	0.9898	0.9898	0.9923	0.9847	0.9954

TABLE 27: Top feature list for RoEduNet-SIMARGL2021 and CICIDS-2017 datasets under SHAP and LIME XAI methods.

RoEduNet-SIMARGL2021 LIME	RF DST_TOS FLOW_DURATION_MILLISECONDS TCP_WIN_MIN_IN TCP_WIN_MAX_IN FLOW_ID TCP_WIN_SCALE_OUT	DNN TCP_WIN_MAX_OUT TCP_WIN_MIN_IN TCP_WIN_MIN_OUT TCP_WIN_SCALE_IN TCP_WIN_SCALE_OUT	LGBM FLOW_DURATION_MILLISECONDS TCP_WIN_MAX_OUT DST_TOS MAX_IP_PKT_LEN LAST_SWITCHED	SVM TCP_WIN_MIN_IN TCP_WIN_SCALE_IN TCP_WIN_MAX_IN TCP_WIN_SCALE_OUT TCP_FLAGS	ADA TCP_WIN_SCALE_IN FIRST_SWITCHED TCP_WIN_MAX_OUT TCP_WIN_MAX_IN TCP_WIN_SCALE_OUT	MLP TCP_WIN_MIN_IN TCP_WIN_SCALE_IN TCP_WIN_MAX_IN TCP_WIN_SCALE_IN TCP_WIN_MIN_IN TCP_WIN_SCALE_IN	KNN TCP_WIN_MIN_IN TCP_WIN_MAX_OUT DST_TOS SRC_TOS TCP_WIN_MAX_IN
RoEduNet-SIMARGL2021 SHAP	RF TCP_WIN_MAX_IN TCP_WIN_SCALE_IN TCP_WIN_MSS_IN TCP_WIN_MIN_IN OUT_PKTS	DNN TCP_WIN_MAX_IN TCP_WIN_MIN_IN TCP_WIN_SCALE_IN L4_SRC_PORT PROTOCOL	LGBM LAST_SWITCHED L4_DST_PORT FIRST_SWITCHED L4_SRC_PORT FLOW_ID	SVM TCP_WIN_MSS_IN TCP_WIN_MAX_IN TCP_WIN_SCALE_IN TCP_WIN_MIN_IN L4_DST_PORT	ADA IN_BYTES TCP_WIN_SCALE_IN PROTOCOL TCP_WIN_MAX_IN L4_SRC_PORT	MLP TCP_WIN_MIN_IN TCP_WIN_SCALE_IN TCP_WIN_MAX_IN TCP_WIN_SCALE_IN TCP_WIN_MIN_IN TCP_WIN_SCALE_IN	KNN TCP_WIN_MAX_IN TCP_WIN_SCALE_IN TCP_WIN_MIN_IN TCP_WIN_SCALE_IN L4_DST_PORT
CICIDS-2017 LIME	RF Bwd Avg Bulk Rate Bwd IAT Total Active Mean Bwd Packet Length Mean Bwd Packet Length Max Min Packet Length FIN Flag Count Bwd PSH Flags min_seg_size_forward Init_Win_bytes_backward Packet Length Std Fwd IAT Max Idle Std Subflow Fwd Packets Packet Length Variance FIN Flag Count Init_Win_bytes_forward Fwd URG Flags Flow Bytes/s Fwd IAT Min SYN Flag Count	DNN Bwd Avg Bulk Rate PSH Flag Count Active Mean Bwd Packet Length Max Idle Std Init_Win_bytes_backward Subflow Bwd Packets Bwd IAT Total Min Packet Length Packet Length Variance Packet Length Std Fwd IAT Min Idle Std min_seg_size_forward Min Packet Length FIN Flag Count Fwd IAT Std Fwd URG Flags Flow IAT Min Idle Min Fwd IAT Mean	LGBM Bwd Packet Length Min Bwd PSH Flags Bwd IAT Min CWE Flag Count Bwd Packets/s Bwd Header Length Bwd Packet Length Mean Packet Length Std Packet Length Mean Subflow Fwd Bytes Subflow Bwd Bytes Fwd Packet Length Min Subflow Bwd Packets Fwd IAT Max Min Packet Length Init_Win_bytes_forward Total Length of Bwd Packets URG Flag Count Active Mean Init_Win_bytes_backward	SVM ACK Flag Count Bwd IAT Total Bwd Avg Bulk Rate Packet Length Variance Packet Length Std Active Mean Init_Win_bytes_backward Bwd Packet Length Std Idle Std min_seg_size_forward Fwd IAT Min Idle Mean Bwd Header Length Packet Length Mean PSH Flag Count Max Packet Length Min Packet Length Subflow Bwd Bytes Fwd PSH Flags Fwd IAT Mean	ADA ACK Flag Count Bwd PSH Flags Bwd Packet Length Mean CWE Flag Count Bwd URG Flags Bwd IAT Min Bwd Packet Length Std Bwd IAT Mean Bwd Avg Bulk Rate Subflow Fwd Packets Bwd IAT Std Bwd Header Length Min Packet Length Fwd IAT Total Init_Win_bytes_backward Idle Std Destination Port Idle Min Fwd IAT Mean	MLP PSH Flag Count Destination Port Init_Win_bytes_forward ACK Flag Count Bwd IAT Total Flow IAT Max min_seg_size_forward Fwd IAT Total URG Flag Count Flow Duration Init_Win_bytes_backward Fwd IAT Max Bwd IAT Max Bwd IAT Std Fwd IAT Std SYN Flag Count Bwd Packets/s Bwd Packet Length Std Packet Length Mean Packet Length Std Average Packet Size	KNN Idle Max Fwd Packet Length Max Bwd IAT Total Fwd URG Flags Fwd Packet Length Min Bwd PSH Flags ACK Flag Count Max Packet Length Fwd Header Length Fwd Avg Bulk Rate Fwd Packet Length Std Bwd Packet Length Min Subflow Bwd Packets Fwd IAT Total Active Std CWE Flag Count Total Backward Packets Packet Length Std Idle Std PSH Flag Count
CICIDS-2017 SHAP	RF Init_Win_bytes_backward Destination Port Bwd Packet Length Std Flow IAT Max Total Length of Fwd Packets Fwd Packet Length Max Flow IAT Mean Fwd Header Length Fwd IAT Std Total Fwd Packets Packet Length Mean Packet Length Variance Active Min Bwd Header Length Flow Duration Packet Length Std Bwd Packet Length Std Fwd IAT Total Bwd Packet Length Max Fwd Packet Length Mean	DNN PSH Flag Count ACK Flag Count Flow Duration Bwd IAT Total Fwd IAT Total Destination Port Init_Win_bytes_backward Idle Max Bwd Packet Length Std Bwd IAT Max Fwd PSH Flags Idle Mean Flow IAT Max Init_Win_bytes_forward Fwd IAT Std URG Flag Count Average Packet Size Subflow Fwd Bytes SYN Flag Count Avg Bwd Segment Size Total Length of Fwd Packets	LGBM Destination Port Init_Win_bytes_backward Bwd IAT Max Bwd IAT Total Init_Win_bytes_forward Bwd Packets/s PSH Flag Count Bwd Packet Length Std Fwd IAT Min Flow IAT Max Fwd Packets/s Flow IAT Mean Fwd Header Length Flow IAT Max Bwd Packet Length Mean Bwd IAT Std Total Fwd Packets Bwd Packet Length Min Flow Duration Packet Length Mean	SVM PSH Flag Count ACK Flag Count min_seg_size_forward Init_Win_bytes_forward Init_Win_bytes_backward Destination Port URG Flag Count Bwd IAT Total Flow Duration Fwd IAT Max Flow IAT Min Bwd IAT Max Fwd IAT Total Idle Max FIN Flag Count Packet Length Std Packet Length Variance Fwd PSH Flags SYN Flag Count Bwd Packet Length Std	ADA Destination Port CWE Flag Count Fwd Avg Packets/Bulk CWE Flag Count Avg Bwd Segment Size Avg Fwd Segment Size Average Packet Size Down/Up Ratio ECE Flag Count URG Flag Count Max Packet Length ACK Flag Count PSH Flag Count RST Flag Count SYN Flag Count FIN Flag Count Idle Std Packet Length Variance Fwd Avg Bulk Rate Bwd Avg Bytes/Bulk	MLP Bwd Packets/s Bwd PSH Flags Bwd Packet Length Mean CWE Flag Count Bwd URG Flags Bwd IAT Min Bwd Packet Length Min Bwd Avg Bulk Rate Subflow Fwd Packets Bwd IAT Std Bwd Header Length Min Packet Length Fwd IAT Total Init_Win_bytes_backward Fwd IAT Std Destination Port Idle Min Fwd IAT Mean	KNN Init_Win_bytes_backward Init_Win_bytes_forward PSH Flag Count min_seg_size_forward ACK Flag Count Fwd IAT Std Flow Duration Flow IAT Std Bwd IAT Total Flow IAT Max Fwd IAT Max Destination Port Fwd Packets/s Flow IAT Mean act_data_pkt_fwd Bwd Packet Length Max Bwd Header Length Bwd Packets/s

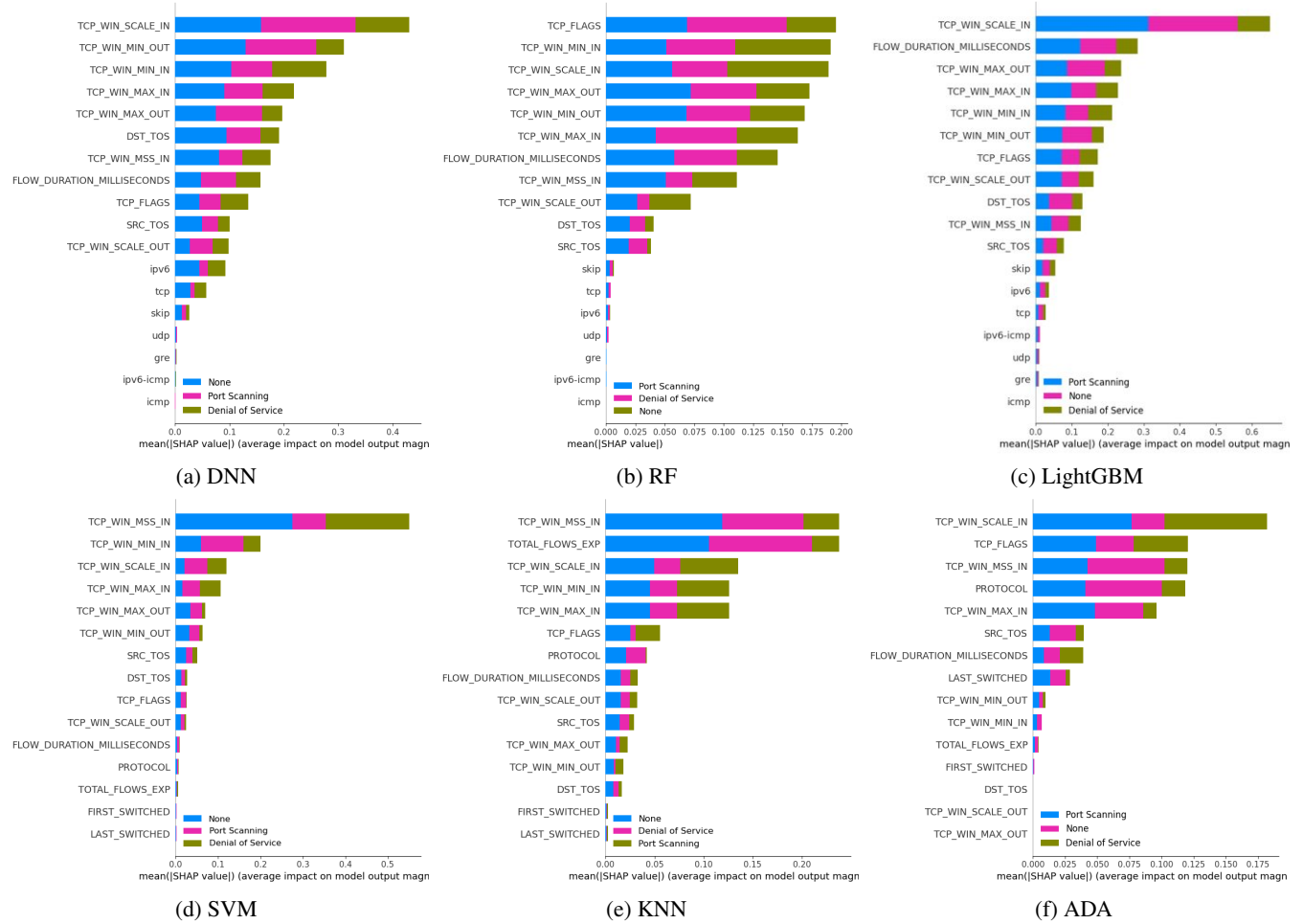


FIGURE 20: Feature importance using global summary plot (by SHAP) for each intrusion detection AI model for the RoEduNet-SIMARGL2021 dataset. It also shows the feature importance for each attack type (different colors).

TABLE 28: Top feature list for NSL-KDD dataset under SHAP and LIME XAI methods for different AI models considered.

NSL-KDD SHAP	ADA	DNN	LGBM	SVM	MLP	RF	KNN
1	dst_host_srv_count	dst_host_error_rate	src_bytes	dst_host_srv_count	dst_host_same_srv_rate	dst_host_error_rate	same_srv_rate
2	dst_host_error_rate	dst_host_same_srv_rate	diff_srv_rate	srv_error_rate	dst_host_srv_count	dst_bytes	flag_SF
3	src_bytes	dst_host_error_rate	dst_host_srv_count	count	dst_host_error_rate	src_bytes	flag_S0
4	dst_bytes	dst_host_srv_count	same_srv_rate	service_http	dst_host_error_rate	flag_S0	dst_host_srv_error_rate
5	count	dst_host_same_src_port_rate	dst_host_same_srv_rate	service_telnet	dst_host_same_src_port_rate	dst_host_same_srv_rate	dst_host_error_rate
6	dst_host_same_src_port_rate	error_rate	dst_host_error_rate	same_srv_rate	root_shell	dst_host_srv_count	error_rate
7	service_http	same_srv_rate	count	dst_host_same_srv_rate	service_telnet	logged_in	srv_error_rate
8	service_private	logged_in	dst_bytes	srv_error_rate	dst_host_count	error_rate	dst_host_same_srv_rate
9	dst_host_diff_srv_rate	dst_host_error_rate	dst_host_same_src_port_rate	dst_host_same_src_port_rate	dst_host_diff_srv_rate	dst_host_diff_srv_rate	dst_host_srv_count
10	dst_host_error_rate	count	flag_S0	dst_host_error_rate	hot	dst_host_error_rate	logged_in
11	dst_host_count	service_http	flag_RSTOS0	hot	same_srv_rate	count	count
12	service_telnet	dst_host_diff_srv_rate	service_http	root_shell	logged_in	flag_SF	service_http
13	duration	dst_host_srv_error_rate	dst_host_error_rate	service_private	flag_REJ	count	dst_host_count
14	srv_count	dst_host_count	logged_in	flag_REJ	count	dst_host_srv_error_rate	dst_host_diff_srv_rate
15	dst_host_srv_diff_host_rate	dst_host_srv_error_rate	Protocol_type_tcp	error_rate	srv_count	same_srv_rate	dst_host_same_src_port_rate
16	error_rate	diff_srv_rate	dst_host_srv_error_rate	logged_in	num_file_creations	service_private	service_telnet
17	flag_S0	srv_error_rate	dst_host_srv_error_rate	dst_host_error_rate	service_http	error_rate	error_rate
18	hot	error_rate	srv_count	dst_host_error_rate	flag_SF	dst_host_same_src_port_rate	root_shell
19	service_other	service_eco_i	service_private	flag_SF	dst_host_diff_srv_rate	diff_srv_rate	service_uucp
20	service_eco_i	flag_SF	service_eco_i	service_ftp_data	dst_host_srv_error_rate	srv_count	num_shells
NSL-KDD LIME	ADA	DNN	LGBM	SVM	MLP	RF	KNN
1	root_shell	service_aol	error_rate	wrong_fragment	wrong_fragment	dst_host_error_rate	service_vmmnet
2	dst_host_error_rate	service_harvest	duration	hot	service_remote_job	error_rate	service_bgp
3	same_srv_rate	service_ftp_u	dst_host_srv_count	service_ecr_i	srv_error_rate	dst_host_srv_error_rate	service_uucp_path
4	service_pm_dump	service_pm_dump	same_srv_rate	root_shell	service_tim_i	dst_host_srv_error_rate	service_courier
5	error_rate	service_ftp_u	src_bytes	is_guest_login	root_shell	service_aol	service_iso_tsap
6	service_tim_i	service_tim_i	wrong_fragment	srv_error_rate	service_ecr_i	service_harvest	service_whois
7	service_domain_u	error_rate	dst_host_error_rate	same_srv_rate	src_bytes	service_red_i	service_csnet_ns
8	srv_error_rate	dst_host_error_rate	service_ftp_u	num_failed_logins	num_shells	service_uh_i	service_nntp
9	service_shell	dst_host_srv_error_rate	service_tim_i	num_root	is_guest_login	service_uh_i	service_daytime
10	service_snmp	dst_host_error_rate	service_shell	srv_count	num_failed_logins	su_attempted	service_hostnames
11	logged_in	service_red_i	service_remote_job	service_sql_net	urgent	service_printer	service_login
12	dst_host_srv_count	wrong_fragment	service_finger	protocol_type_icmp	count	service_rje	service_uucp
13	hot	hot	dst_host_count	dst_host_error_rate	dst_host_count	service_ntp_u	service_efs
14	service_pm_dump	num_shells	service_aol	service_kshell	service_other	service_pop_u	service_netbios_dgm
15	service_telnet	count	dst_host_error_rate	service_nttp	dst_host_same_srv_rate	flag_u	service_sysstat
16	dst_host_srv_error_rate	su_attempted	dst_host_error_rate	service_uucp	dst_host_error_rate	service_sql_net	service_exec
17	num_file_creations	root_shell	dst_host_error_rate	dst_host_error_rate	service_ftp	num_file_creations	service_discard
18	dst_host_error_rate	service_rje	dst_host_error_rate	error_rate	dst_host_error_rate	num_file_creations	wrong_fragment
19	error_rate	num_root	dst_host_error_rate	dst_host_count	num_access_files	service_ssh	service_name
20	dst_host_error_rate	service_domain	dst_host_diff_srv_rate	service_klogin	service_harvest	service_kshell	service_klogin

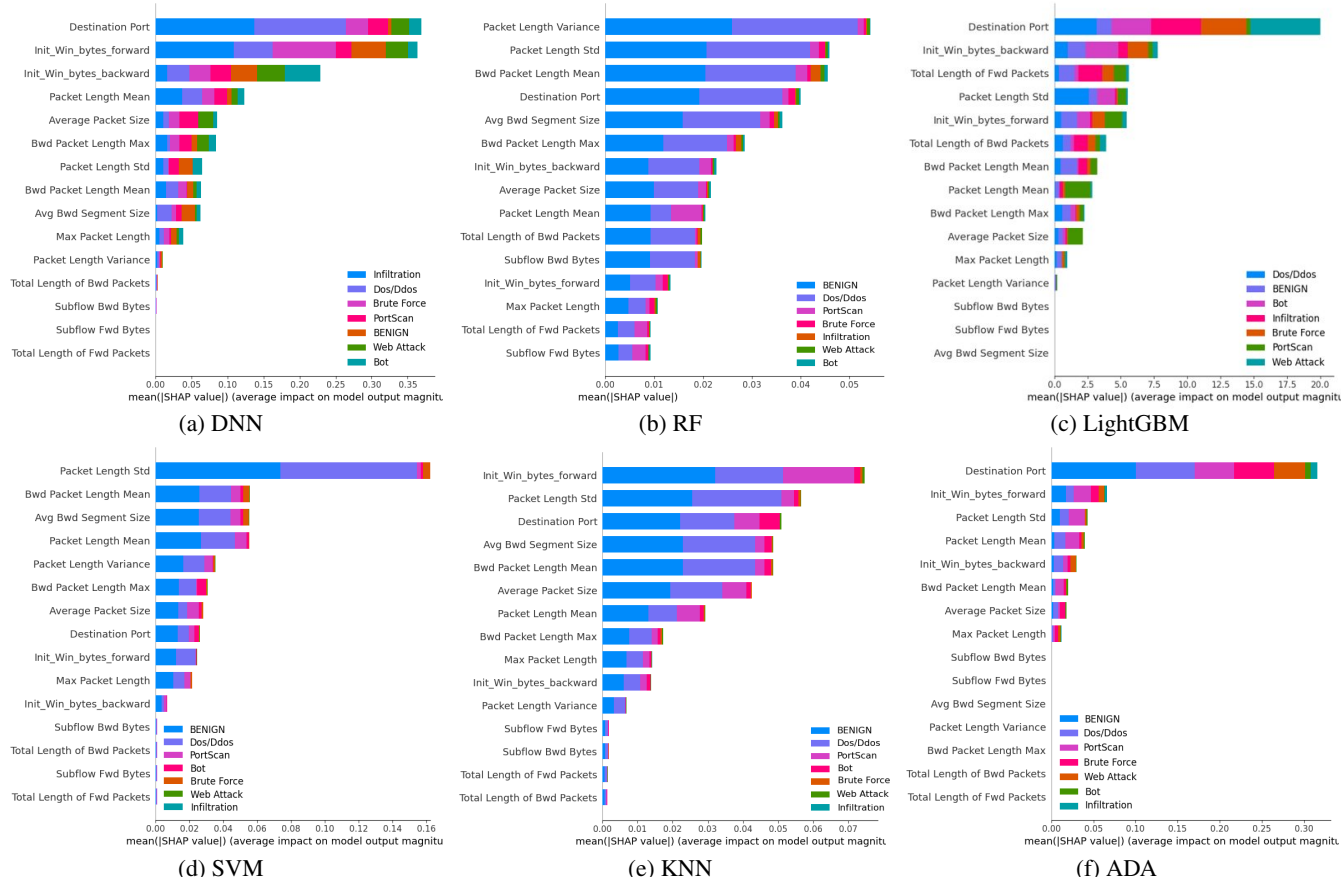


FIGURE 21: Feature importance using global SHAP summary plot for each intrusion detection model for CICIDS-2017 dataset.

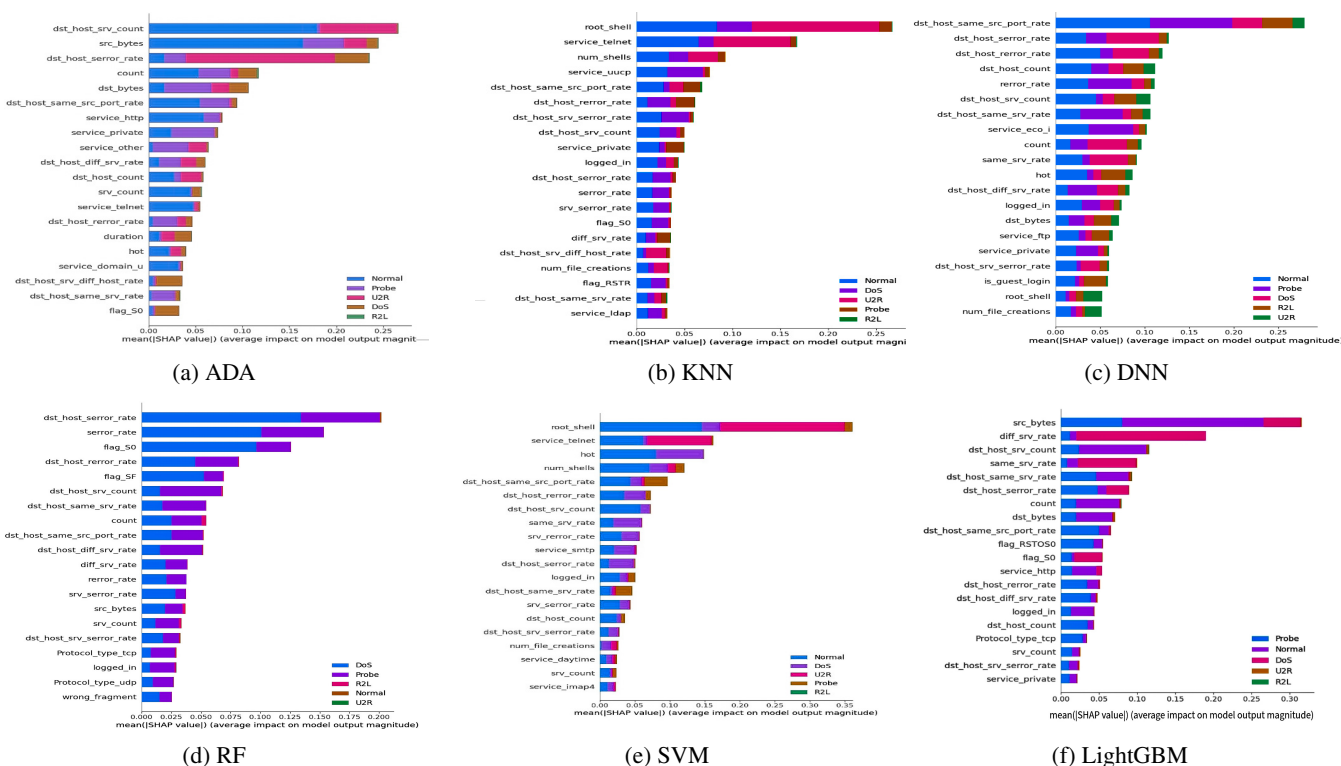


FIGURE 22: Feature importance using global summary plot (by SHAP) for each intrusion detection AI model for the NSL-KDD.



Osvaldo ARRECHE is a Master's student at Purdue School of Engineering and Technology at Indiana University-Purdue University Indianapolis (IUPUI), where he is pursuing a degree in Electrical and Computer Engineering. He is currently working as a Research Assistant focusing in cybersecurity and Explainable AI. He also holds a Bachelor's in Instrumentation, Automation, and Robotics from the Federal University of ABC (UFABC) in Santo Andre, Brazil. He has several industry and manufacturing experience, having spent 3 years as an automation engineer in food and pharma manufacturing. Osvaldo has hands-on experience with technology's practical applications. He aspires to leverage his expertise in creating transparent and ethically-driven technological advancements. Contact him at oarreche@iu.edu or oarreche@purdue.edu.



MUSTAFA ABDALLAH is an Assistant Professor at the Purdue School of Engineering and Technology at Indiana University-Purdue University Indianapolis (IUPUI). Dr. Abdallah got his Ph.D. from the Elmore Family School of Electrical and Computer Engineering at Purdue University. He got the M.Sc. degree in Engineering Mathematics from the Faculty of Engineering at Cairo University. Dr. Abdallah's research interests include game-theory, human decision-making, explainable AI, and machine-learning with applications including network security, and autonomous driving systems. Dr. Abdallah has also several industrial research experiences, including internships with Adobe Research, and Principal, and a five-year machine learning research experience with RDI (a leading machine learning company in Egypt). Dr. Abdallah's research contribution is recognized by receiving the Purdue Bilsland Dissertation Fellowship and having many publications in top IEEE/ACM journals and conferences. He also was the recipient of a M.Sc. Fellowship from the Faculty of Engineering at Cairo University in 2013. Contact him at mabdall@iu.edu

...



TANISH GUNTUR is currently a student in the Computer and Information Science department at Indiana University-Purdue University Indianapolis (IUPUI). He is in his 3rd semester and anticipates graduating in May 2024. Tanish's research interests span a variety of areas including explainable artificial intelligence (XAI), intrusion detection systems, computer vision, image processing, cybersecurity, and data science. He had the opportunity to intern at ICU Medical, gaining valuable experience in his field. Prior to his current studies at IUPUI, Tanish earned a bachelor's degree in Electronics and Communication Engineering from Amrita School of Engineering in 2022. Contact him at tguntur@iu.edu.



JACK ROBERTS is a Cybersecurity Undergraduate student at the Purdue School of Engineering and Technology at Indiana University-Purdue University Indianapolis (IUPUI). Jack is a current participant in IUPUI's 1st Year Research Immersion Program where he has the opportunity to gain important research, problem solving, and critical thinking skills. His interests include cybersecurity, information security and, network design. He plans on continuing his studies in cybersecurity and hopes to gain valuable experience during his time at IUPUI. Contact him at roberjaw@iu.edu